

# Residual-Slack Compatible Descendant Completeness for Locked NAND

A machine-checkable certificate report for  $P = NP$  over the locked NAND residual-slack package

Residual-band exact minimization, locked NAND SAT embedding, traceable normalization, saturated support calculus, projection-defect cellization, hereditary and budget closure, selector realization, and generated proof-package checking

Complete machine-checkable proof report.

## Accepted proof-report boundary.

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \implies P = NP.$$

This paper records the completed finite-certificate route and its executable release seal. The repository emits an accepted `FinalPNPProofReport0` whose theorem field is  $P = NP$ , whose antecedent is `CheckPCCPackexp(GeneratePCCPack()) = accept`, and whose final acceptance/replay, final certificate, release gate, release audit, and proof report records all accept. The hardened validation log for the sealed repository state reports 984 passing tests, no failures, no cancellations, and an accepted final proof-report summary and full check record. Release custody separates the source/checker revision from the durable artefact-bearing release: source tag `final-pnp-proof-report-hardened-105af51`; sealed artefact tag `final-pnp-proof-report-artifacts-hardened-105af51-sealed`. The materialized proof-report artefact bundle is committed under [proof-artifacts/final-pnp-proof-report-hardened-105af51/](#); its manifest is [proof-artifacts/final-pnp-proof-report-hardened-105af51/release-seal.json](#), and non-self file-level checksums are recorded in [proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS](#). If repository access is restricted, request access from [pnp@keaton.com.au](mailto:pnp@keaton.com.au).

# Contents

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Executive status and claim boundary</b>                         | <b>4</b>  |
| 1.1       | Main message . . . . .                                             | 4         |
| 1.2       | Claim boundary . . . . .                                           | 4         |
| 1.3       | Central scale correction . . . . .                                 | 5         |
| <b>2</b>  | <b>Base direct-wire framework</b>                                  | <b>5</b>  |
| 2.1       | Direct-wire words . . . . .                                        | 5         |
| 2.2       | Compatible supports and replacement . . . . .                      | 5         |
| <b>3</b>  | <b>Saturated support calculus and square closure</b>               | <b>6</b>  |
| 3.1       | Raw, completed, and saturated supports . . . . .                   | 6         |
| 3.2       | Support squares and projection compatibility . . . . .             | 6         |
| <b>4</b>  | <b>Global charge and materializer ownership</b>                    | <b>7</b>  |
| 4.1       | Chargeable objects . . . . .                                       | 7         |
| 4.2       | Touch-to-charge principle and charge laws . . . . .                | 7         |
| <b>5</b>  | <b>Mode projection calculus</b>                                    | <b>7</b>  |
| 5.1       | Full and quotient modes . . . . .                                  | 7         |
| 5.2       | Mode firewall and transfer identity . . . . .                      | 8         |
| <b>6</b>  | <b>Verifier-readable proof calculus, normalization, and splice</b> | <b>8</b>  |
| 6.1       | Package E: direct-wire verifier . . . . .                          | 8         |
| 6.2       | Traceable normalization . . . . .                                  | 9         |
| 6.3       | Bounded and composed splices . . . . .                             | 9         |
| <b>7</b>  | <b>Finite local routers: FT, X, BC, and UN</b>                     | <b>9</b>  |
| 7.1       | Finite-table engine FT . . . . .                                   | 9         |
| 7.2       | Critical-window tables X and FT-X . . . . .                        | 10        |
| 7.3       | Branch-cycle router BC . . . . .                                   | 10        |
| 7.4       | Unary decoder UN . . . . .                                         | 10        |
| <b>8</b>  | <b>Hereditary normal forms, HResolve, and BUD</b>                  | <b>11</b> |
| 8.1       | Hereditary package HN . . . . .                                    | 11        |
| 8.2       | Global hereditary resolver . . . . .                               | 11        |
| 8.3       | Budget resolver . . . . .                                          | 11        |
| <b>9</b>  | <b>Frontier-faithful comparison and strong neutral overlay</b>     | <b>12</b> |
| 9.1       | Strong neutral overlay . . . . .                                   | 12        |
| 9.2       | FF-LOSS and frontier-faithful comparison . . . . .                 | 12        |
| <b>10</b> | <b>Residual witness normal form</b>                                | <b>13</b> |
| 10.1      | Terminal whole-carrier bridge . . . . .                            | 13        |
| 10.2      | Full-span policy . . . . .                                         | 13        |
| 10.3      | SaturatePositive and RankWF . . . . .                              | 13        |
| 10.4      | BCEL-ready residual witnesses . . . . .                            | 13        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>11 BN2 through BN6 and Package C</b>                              | <b>14</b> |
| 11.1 BN2: side-tight coherent optima . . . . .                       | 14        |
| 11.2 BN3: simultaneous finite request envelopes . . . . .            | 15        |
| 11.3 BN4: activation-exact cancellation . . . . .                    | 15        |
| 11.4 BN5: full-shadow localization . . . . .                         | 15        |
| 11.5 Package C, renamed PkgC: separating consumers . . . . .         | 16        |
| 11.6 BN6: hypergraph cellization and packet collapse . . . . .       | 16        |
| <b>12 Finite rigidity and consumer antichains</b>                    | <b>16</b> |
| <b>13 Packet extraction and selector coverage</b>                    | <b>17</b> |
| 13.1 Selector universe . . . . .                                     | 17        |
| 13.2 Pair packet extraction . . . . .                                | 17        |
| 13.3 Balanced-triple extraction . . . . .                            | 18        |
| 13.4 Full-span spine extraction . . . . .                            | 18        |
| <b>14 Package R: selector realization and strict surplus</b>         | <b>18</b> |
| 14.1 Public realizer contract . . . . .                              | 18        |
| 14.2 Strict charge surplus . . . . .                                 | 19        |
| <b>15 Simultaneous HN-BUD rank-negative closure</b>                  | <b>19</b> |
| <b>16 Oracle, ZeroSlack, and residual-band minimization</b>          | <b>20</b> |
| 16.1 Rank-ordered PCCOracle . . . . .                                | 20        |
| 16.2 PCCMin . . . . .                                                | 20        |
| <b>17 Locked NAND SAT embedding</b>                                  | <b>21</b> |
| 17.1 Carrier slots and invariants . . . . .                          | 21        |
| 17.2 Macros . . . . .                                                | 21        |
| 17.3 Baseline and threshold . . . . .                                | 22        |
| 17.4 Output convention and threshold proof details . . . . .         | 22        |
| <b>18 Proof kernels, package manifest, and final theorem</b>         | <b>24</b> |
| 18.1 PCC-K and PCC-K Sigma . . . . .                                 | 24        |
| 18.2 Interface dictionary and package manifest . . . . .             | 24        |
| 18.3 CheckPack . . . . .                                             | 25        |
| 18.4 Final SAT decision . . . . .                                    | 25        |
| <b>19 Schema generation, interface consistency, and normal forms</b> | <b>26</b> |
| 19.1 Finite package generation contract . . . . .                    | 26        |
| 19.2 No-hidden-minimization scan . . . . .                           | 26        |
| 19.3 Interface dictionary . . . . .                                  | 26        |
| 19.4 Normal forms and row keys . . . . .                             | 26        |
| 19.5 Schema inventory status . . . . .                               | 27        |
| <b>20 Integrated completion contracts and artefact stack</b>         | <b>27</b> |
| 20.1 Verifier-level package completions . . . . .                    | 27        |
| 20.2 Bounded parameter schedule . . . . .                            | 28        |
| 20.3 Concrete row-generation protocol . . . . .                      | 29        |

|                                                                                   |           |
|-----------------------------------------------------------------------------------|-----------|
| 20.4 PCC-K and PCC-K Sigma implementation . . . . .                               | 30        |
| 20.5 Checker hardening . . . . .                                                  | 30        |
| 20.6 Locked NAND and final integration artefacts . . . . .                        | 31        |
| 20.7 Final acceptance run . . . . .                                               | 31        |
| 20.8 Bootstrap executable seed . . . . .                                          | 32        |
| 20.9 PCC-K, PCC-K Sigma, and reflection implementation . . . . .                  | 32        |
| 20.10 Hardened checker suite . . . . .                                            | 32        |
| 20.11 Concrete row generation . . . . .                                           | 32        |
| 20.12 Global proof DAG . . . . .                                                  | 33        |
| 20.13 Locked NAND and final integration . . . . .                                 | 33        |
| 20.14 Final acceptance run and deterministic generator . . . . .                  | 33        |
| 20.15 Byte-level skeleton package . . . . .                                       | 34        |
| 20.16 First bootstrap row batch . . . . .                                         | 34        |
| 20.17 Concrete <i>KernelSeed</i> <sub>0</sub> . . . . .                           | 34        |
| 20.18 Concrete <i>Codec</i> <sub>0</sub> and <i>Digest</i> <sub>0</sub> . . . . . | 34        |
| 20.19 First runnable verifier fragments . . . . .                                 | 35        |
| 20.20 Final implementation boundary . . . . .                                     | 35        |
| 20.21 Accepted final artefact stack . . . . .                                     | 35        |
| 20.22 Reproducibility and repository access . . . . .                             | 36        |
| <b>21 Formula sheet</b>                                                           | <b>36</b> |
| <b>22 Theorem ledger</b>                                                          | <b>37</b> |
| <b>23 Final proof-report release seal</b>                                         | <b>37</b> |
| 23.1 Sealed revision and validation . . . . .                                     | 38        |
| 23.2 Accepted report fields . . . . .                                             | 38        |
| 23.3 Digest ledger . . . . .                                                      | 39        |
| 23.4 Final theorem extraction . . . . .                                           | 39        |
| 23.5 Independent verification protocol . . . . .                                  | 39        |
| <b>24 Appendix A: locked NAND macro truth signatures</b>                          | <b>40</b> |
| 24.1 Equality macro $M_{=}$ . . . . .                                             | 40        |
| 24.2 Constant macros . . . . .                                                    | 40        |
| 24.3 NAND trace-check macro $M_N$ . . . . .                                       | 40        |
| <b>25 Appendix B: checklist for package acceptance</b>                            | <b>41</b> |
| <b>26 Closing summary</b>                                                         | <b>42</b> |

# 1 Executive status and claim boundary

## 1.1 Main message

This paper proves

$$P = NP$$

by reducing SAT to exact minimization of a specially locked multi-output NAND direct-wire word with constant residual slack, proving polynomial residual-band exact minimization by a finite proof-carrying package, and checking the generated package through a deterministic executable acceptance and replay stack.

The proof route is organized around the implication

$$\begin{aligned} &\text{accepted PCCPack over locked NAND is valid and polynomial} \\ &\implies \text{polynomial exact residual-band minimization} \\ &\implies SAT \in P \implies P = NP. \end{aligned}$$

The final generated-package certificate is

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept},$$

and the final theorem recorded by the accepted proof report is

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \implies P = NP.$$

The release custody for this proof report separates source/checker provenance from the durable artefact bundle. The hardened source/checker revision is the annotated git tag

`final-pnp-proof-report-hardened-105af51`

at commit

`105af516128fa0f7cc9978e6381bb6d8afdc7058`.

The generated artefact JSON commit before metadata sealing is

`82e1a234a5a589c96c64dc12b2dd285e4b7d9b30`.

The durable sealed artefact release is the annotated git tag

`final-pnp-proof-report-artifacts-hardened-105af51-sealed`.

The materialized proof-report bundle is

`proof-artifacts/final-pnp-proof-report-hardened-105af51/`.

## 1.2 Claim boundary

The claim boundary is the accepted, replayable finite artefact stack culminating in `CheckFinalPNPProofReport0`. The mathematical theorem is asserted under the explicit checker trust model: the finite package, acceptance run, replay, final certificate, release gate, and proof report are the objects being checked, and the public claim is made only because the final proof report accepts.

**Theorem 1.1** (Final accepted finite-certificate theorem). *Let  $P_{\text{gen}} = \text{GeneratePCCPack}()$ . In the sealed revision, the executable artefact stack records*

$$\text{CheckPCCPack}_{\text{exp}}(P_{\text{gen}}) = \text{accept}, \quad \text{CheckAcceptRun}_{\text{exp}}(P_{\text{gen}}) = \text{accept}, \quad \text{ReplayAcceptRun}_{\text{exp}}(P_{\text{gen}}) = \text{accept},$$

*and the final proof report accepts the public theorem statement*

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \implies P = NP.$$

*Therefore SAT is decidable in polynomial time and  $P = NP$ .*

*Proof.* Sections 2–19 prove the locked NAND residual-slack certificate route: an accepted explicit package yields polynomial exact residual-band minimization and hence SAT in polynomial time. Sections 20 and 23 identify the generated package, deterministic acceptance run, replay, final certificate, release gate, and proof-report artefacts. The final accepted report records accepted `CheckPCCPackexp0`, accepted `CheckAcceptRun0`, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, accepted final release gate, and accepted proof report. The report's theorem field has antecedent `CheckPCCPackexp(GeneratePCCPack()) = accept`, consequent  $P = NP$ , and conditional flag true. Applying the package-sufficiency theorem to the accepted generated package gives  $SAT \in P$ . Since SAT is NP-complete,  $P = NP$ .  $\square$

### 1.3 Central scale correction

The descent scale is residual slack, not root circuit size. For a circuit  $C$ , define

$$\Lambda(C) = |C| - \mu(C),$$

where  $\mu(C)$  is the minimum equivalent circuit size in the chosen framework. For a compatible support  $U \subseteq C$ , let  $F_{C,U}$  be the induced open carrier-tagged function and define

$$g_C(U) = |U| - \mu^*(F_{C,U}).$$

The global slack law is

$$g_C(U) \leq \Lambda(C).$$

Thus one verified unit gain decreases residual slack by at least one. Locked NAND SAT instances constructed below satisfy

$$\Lambda(W_\varphi^{NAND}) \leq 4.$$

## 2 Base direct-wire framework

### 2.1 Direct-wire words

A carrier is a tuple

$$L = (\partial L, \iota L, \text{prof } L),$$

where  $\partial L$  is an ordered boundary tuple,  $\iota L$  is an ordered interface or output tuple, and  $\text{prof } L$  records carrier constants, prefix-tail slots, origin slots, finite-kernel slots, obligation slots, direction records, saturation records, budget records, and charge slots.

A NAND direct-wire word in carrier  $L$  is a sequence

$$W = (g_1, \dots, g_m, o),$$

where each gate is

$$g_j = \mathbf{N}(s_j, t_j) = \neg(s_j \wedge t_j),$$

and each source is a boundary coordinate, carrier constant 0 or 1, or an earlier gate output. The output tuple  $o$  is ordered.

The semantics map is

$$[[ - ]]_L : \text{DWWord}(L) \rightarrow \text{OpenFun}(L).$$

For an open function  $F$ , define

$$\mu_L^*(F) = \min\{|W| : [[W]]_L = F\}.$$

### 2.2 Compatible supports and replacement

A set  $U \subseteq C$  is compatible if every wire entering  $U$  from outside is listed as a boundary port, every wire leaving  $U$  is listed as an interface port, and all internal gate sources are internal to  $U$  or listed boundary sources. Extraction gives

$$(L_U, W_U) = \text{Extract}(C, U),$$

with

$$[[W_U]] = F_{C,U}.$$

**Lemma 2.1** (Compatible replacement). *If  $U$  is compatible and  $R$  is an open word in the same carrier with  $[[R]] = F_{C,U}$ , then replacing  $U$  by  $R$  preserves the global function of  $C$ .*

**Lemma 2.2** (Global slack law). *For every compatible support  $U \subseteq C$ ,*

$$g_C(U) \leq \Lambda(C).$$

*Proof.* Replace  $U$  by a minimum open realization of  $F_{C,U}$ . The resulting circuit computes the same global function and has size  $|C| - g_C(U)$ . Hence  $\mu(C) \leq |C| - g_C(U)$ , which rearranges to the claim.  $\square$

### 3 Saturated support calculus and square closure

#### 3.1 Raw, completed, and saturated supports

Let  $\mathcal{P}(C)$  be the primitive record universe of  $C$ : NAND gates, boundary and interface records, carrier records, origin records, kernel records, obligation records, prefix-tail records, budget records, direction records, saturation records, and charge records.

A raw support is a finite subset

$$U^{raw} \subseteq \mathcal{P}(C).$$

A completed support is

$$\text{Comp}(U^{raw}) = (U, \partial U, \iota U, \mathcal{F}_U),$$

where completion records incoming wires as boundary coordinates, outgoing wires as ordered interface coordinates, and every touched profile record as internal, materialized, or exported as a frontier obligation.

Define a finite closure operator

$$\text{Sat}_C : \mathcal{P}(C) \rightarrow \mathcal{P}(C).$$

It is generated by gate-source closure, interface-consumer closure, origin closure, finite-kernel closure, obligation closure, prefix-tail closure, budget and saturation closure, direction closure, and charge closure.

**Theorem 3.1** (Saturation closure). *For all  $U, V \subseteq \mathcal{P}(C)$ ,*

$$U \subseteq \text{Sat}_C(U), \quad \text{Sat}_C(\text{Sat}_C(U)) = \text{Sat}_C(U),$$

and

$$U \subseteq V \implies \text{Sat}_C(U) \subseteq \text{Sat}_C(V).$$

For saturated supports  $A, B$ , raw union and intersection are completed by

$$A \vee B = \text{Comp}(\text{Sat}_C(A \cup_{raw} B)),$$

$$A \wedge B = \text{Comp}(\text{Sat}_C(A \cap_{raw} B)).$$

The notation  $A \cup B$  and  $A \cap B$  is legal in theorem statements only after this saturation and completion step.

**Theorem 3.2** (Saturated support square closure). *If  $A, B$  are compatible saturated supports, then  $A \wedge B$  and  $A \vee B$  are compatible saturated supports, unless completion exposes a first obstruction of kind CritC,  $Q, E, L$ , or a budget, hereditary, selector,  $X$ -failure, verified gain, exact route, or strict descent outcome.*

#### 3.2 Support squares and projection compatibility

The accepted support square is

$$\begin{array}{ccc} A \wedge B & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & A \vee B. \end{array}$$

The frontier pushout condition is

$$\mathcal{F}_{A \vee B} = \mathcal{F}_A \star_{\mathcal{F}_{A \wedge B}} \mathcal{F}_B,$$

with carrier, origin, kernel, obligation, prefix-tail, direction, saturation, budget, and charge records glued by explicit transport. If projection does not commute with the square, the first failure routes to CritC,  $Q, E, L$ , or to semantic, direction, resource, Hall, budget, selector, exact-route, or descent outcomes.

**Theorem 3.3** (BN2 square legitimacy). *Every BN2 support square in a no-outcome branch is a legitimate saturated projection-compatible square. Thus  $m_i(A), m_i(B), m_i(A \wedge B), m_i(A \vee B), \delta_i(A, B), \Omega(A, B)$ , and  $d(A)$  are well-defined on the same carrier-compatible square.*

## 4 Global charge and materializer ownership

### 4.1 Chargeable objects

For a descendant  $C$ , define the charge universe  $\text{Ch}(C)$ . Allowed chargeable objects are physical NAND gates, exact-prefix materializers, origin materializers, finite-kernel materializers, closed obligation materializers, and charged profile contexts. Ordinary gates have weight 1. A materializer record  $\rho$  has weight

$$\text{wt}(\rho) = |\text{Mat}(\rho)|.$$

Proof-only records are not in  $\text{Ch}(C)$  and cannot feed gates or appear as free Boolean outputs.

The size convention is

$$|C| = \sum_{x \in \text{Ch}(C)} \text{wt}(x).$$

Every chargeable object has a unique owner

$$\text{own}_C : \text{Ch}(C) \rightarrow \text{Own}(C).$$

### 4.2 Touch-to-charge principle and charge laws

For every normalized profile record  $\rho$ , a touch predicate  $\text{Touch}_C(U, \rho)$  is defined. If a support touches a computational profile record, then the record must be a genuine free carrier datum, materialized and charged inside the support, materialized and charged on the exterior frontier, or represented as an open obligation that cannot be used until discharged.

For a traceable normalization step  $C_t \rightarrow C_{t+1}$ ,

$$\kappa_t(W) = \sum_{\rho \in \text{Touch}_t(W)} \text{wt}(\text{Mat}_t(\rho)),$$

and for every support and replacement,

$$|\text{Pull}_t(W)| - |W| = |\text{Expand}_t(R)| - |R| = \kappa_t(W).$$

For an HN leaf  $L$  with blocks  $B_i$ ,

$$\text{Ch}(U) = \text{Ch}_{\text{fixed}}(L) \sqcup \bigsqcup_i \text{Ch}(B_i),$$

so a BWL path  $p$  satisfies

$$|R_p| = \kappa_L + \sum_{e \in p} \text{cost}(e).$$

**Theorem 4.1** (Charge soundness). *If  $\text{CheckCharge}_{\text{exp}}$  accepts a normalization, HN, budget, splice, BCEL, selector, or neutral-overlay charge certificate, then the relevant size equation is an exact integer identity over uniquely owned computational charges.*

## 5 Mode projection calculus

### 5.1 Full and quotient modes

A full carrier is

$$L^1 = (\partial, \iota, \text{prof}^1),$$

with

$$\text{prof}^1 = (\text{car}, \text{orig}, \text{ker}, \text{obl}, \text{pref}, \text{dir}, \text{sat}, \text{bud}, \text{chg}, \text{front}).$$

A quotient carrier is

$$L^0 = \rho L^1,$$

where  $\rho$  may forget or coarsen finite profile data. It does not add free Boolean functions.

For a full word  $W^1$ , projection  $\rho W^1$  keeps the same NAND DAG and applies  $\rho$  to profile references. Therefore

$$|\rho W^1| = |W^1|,$$

and

$$[[\rho W^1]]_{L^0} = \rho([W^1]_{L^1}).$$

For a saturated support  $X$ , define

$$F_X^0 = \rho F_X^1, \\ m_1(X) = \mu_1^*(F_X^1), \quad m_0(X) = \mu_0^*(F_X^0),$$

and projection defect

$$d(X) = m_1(X) - m_0(X).$$

**Theorem 5.1** (Projection monotonicity). *For every saturated support  $X$ ,*

$$m_0(X) \leq m_1(X), \quad d(X) \geq 0.$$

## 5.2 Mode firewall and transfer identity

A quotient equality is not a full equality. A quotient word can be used constructively in full mode only if it has a full lift and all lost-bit or finite-kernel obligations are discharged through Package E. Otherwise it remains a comparison object used to measure projection defect.

For projection-compatible saturated support squares, define

$$\delta_i(X, Y) = m_i(X) + m_i(Y) - m_i(X \wedge Y) - m_i(X \vee Y), \\ \Omega(X, Y) = \delta_0(X, Y) - \delta_1(X, Y).$$

Then

$$\boxed{d(X \vee Y) + d(X \wedge Y) = d(X) + d(Y) + \Omega(X, Y).}$$

## 6 Verifier-readable proof calculus, normalization, and splice

### 6.1 Package E: direct-wire verifier

A constructive gain certificate is

$$(C, U, R, \Pi),$$

where  $U \subsetneq C$  is compatible,  $R$  is an explicit replacement open word, and  $\Pi$  proves

$$[[R]] = F_{C,U}, \quad |R| \leq |U| - 1.$$

The rewrite families are R1 to R9: structural congruence, finite carrier recoding, exact-prefix strip and absorb, latent sharing and fold, finite-kernel bit drop with obligation creation, matched cancellation with full-mode discharge, unary cut realization, origin pullback and promoted latent expansion, and integer incidence accounting.

Only R5 creates obligations. Only R6, R7, and R8 discharge obligations. Final ledgers must have no open obligations. R6 may discharge an obligation only with a full-mode cancellation witness. Projected equality alone is insufficient.

**Theorem 6.1** (VerifyDW soundness). *If*

$$\text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept},$$

*then  $U \subsetneq C$  is compatible,*

$$[[R]] = F_{C,U}, \quad |R| \leq |U| - 1.$$

*Consequently,*

$$[[C[U \leftarrow R]]] = [[C]], \quad \Lambda(C[U \leftarrow R]) \leq \Lambda(C) - 1.$$

## 6.2 Traceable normalization

The normalizer uses traceable operations N1 to N10: topological reorder and renaming, dead bookkeeping deletion, carrier recoding, tuple normalization, exact-prefix metadata absorption, origin normalization, finite-kernel normalization, obligation sorting and cancellation, profile normalization, and gain surfacing.

A step  $C_t \rightarrow C_{t+1}$  carries

$$Tr_t = (\Phi_t, Pull_t, Expand_t, \kappa_t, \Pi_t).$$

For every compatible support  $W \subseteq C_{t+1}$  and replacement  $R$ ,

$$|Pull_t(W)| - |W| = |Expand_t(R)| - |R|.$$

Thus strict saving transports backward.

**Theorem 6.2** (Traceable normalization). *If  $NormalizeOrGain_{\text{exp}}(C) = Normal(C^\nu, T^\nu)$ , then*

$$[[C^\nu]] = [[C]], \quad |C^\nu| \leq |C|,$$

$$\mu(C^\nu) = \mu(C), \quad \Lambda(C^\nu) \leq \Lambda(C).$$

*Every positive residual witness in  $C^\nu$  pulls back to a positive residual witness in  $C$ . If  $NormalizeOrGain_{\text{exp}}(C) = Gain(U, R, \Pi)$ , then  $VerifyDW_{\text{exp}}(C, U, R, \Pi) = \text{accept}$ .*

## 6.3 Bounded and composed splices

A bounded splice instance consists of a compatible bounded support  $U$ , a same-frontier replacement  $R$ , symbolic windows  $\omega_U, \omega_R$ , finite transport, materializer ledger, obligation-discharge ledger, and size ledger. It requires ordered frontier equality, bounded truth-vector equality, finite relation equality, profile transport, full-mode obligation closure, and

$$|R| \leq |U| - 1.$$

**Theorem 6.3** (Bounded splice). *Every accepted bounded splice instance compiles to a ledger  $\Pi$  with*

$$VerifyDW_{\text{exp}}(C, U, R, \Pi) = \text{accept}.$$

A composed splice decomposes a long support as

$$U = U_1 \cup \dots \cup U_m$$

with declared seams. Seam compatibility requires ordered interface-boundary agreement, carrier agreement, origin and kernel agreement, full-mode obligation compatibility, direction and saturation agreement, and unique budget and charge ownership.

**Theorem 6.4** (Composed splice). *If a composed splice instance is seam-compatible, charge-valid, and at least one local or seam component has strict saving, then the generated global ledger satisfies  $VerifyDW_{\text{exp}}$ .*

## 7 Finite local routers: FT, X, BC, and UN

### 7.1 Finite-table engine FT

A finite local table is built over symbolic windows

$$\omega = (D, \partial, \iota, o, H, prof, guard, relSpec, payloadSlots),$$

where  $D$  is a bounded NAND DAG,  $H$  is a finite handle-variable tuple, and  $prof$  lies in finite profile alphabets. A schema is

$$S = (k, \beta, o_{max}, h, \mathcal{P}, \mathcal{A}, \mathcal{G}, \mathcal{C}, NF, PayloadSpec).$$

The raw universe is finite. Runtime integers such as  $D, p, q, M_0, B_1$  are arithmetic fields, not finite profile states.

The bounded evaluator enumerates all  $2^\beta$  formal boundary assignments. Symbolic normalization returns both a code and transport:

$$NF(\omega) = (\hat{\omega}, \tau_\omega).$$

The transport preserves truth vectors, relation tables, profile fields, handle slots, output order, source incidence, consumer incidence, dependency edges, charge ownership, obligations, direction, saturation, budget, and payload fields.

**Theorem 7.1** (Finite-table coverage). *If  $CheckTable_{\text{exp}}(T, S) = \text{accept}$ , then every physical configuration governed by  $S$  is represented, normalized, classified, and soundly routed by  $T$ .*

## 7.2 Critical-window tables X and FT-X

The finite critical kinds are:

$$\begin{aligned} \text{CritC} &= \text{full row seeking quotient shadow,} \\ Q &= \text{quotient row seeking full restoration,} \\ E &= \text{exposure-obligation retract,} \\ L &= \text{overlap-loss preservation.} \end{aligned}$$

Each row has candidate towers

$$\text{ResTab}_\kappa(r) \subseteq \text{DirTab}_\kappa(r) \subseteq \text{SemTab}_\kappa(r).$$

Empty semantic, direction, or resource candidate sets produce  $X2, X3, X4$ , respectively, with exhaustive certificates. If resource targets exist, matching is performed fibrewise by projected signature. A Hall witness gives  $X1$ . A bounded strict replacement emits  $A(s)$  and compiles to  $\text{VerifyDW}_{\text{exp}}$ .

The deterministic route priority for X rows is

$$A(s) \succ X2 \succ X3 \succ X4/\text{CritKind} \succ X1 \succ \text{CritKind} \succ \text{Neutral} \succ \text{Downstream}.$$

This prevents a row with a bounded gain from being downgraded to an obstruction, and prevents semantic impossibility from masquerading as a weaker kind-specific row.

## 7.3 Branch-cycle router BC

Package BC handles connected full-span non-unary incidence graphs. It uses a finite transition category with states in

$$\Sigma_{BC} = \Sigma_{car} \times \Sigma_{pref} \times \Sigma_{orig} \times \Sigma_{ker} \times \Sigma_{obl} \times \Sigma_{dir} \times \Sigma_{sat} \times \Sigma_{bud} \times \Sigma_{chg}.$$

Transition labels include full and projected transition relations, projection record, defect, saturation, budget, obligation, charge, size, normalized code, and local certificate.

Composition is partial. Undefined composition emits a first failed coordinate, such as  $X2, X3, X4, \text{CritC}, Q, E, L$ , budget, HNShape, graph descent, or UnaryWindow. Cycle holonomy and branch tripod CSPs are finite.

**Theorem 7.2** (Package BC). *Every governed connected full-span non-unary primitive critical window routes to a verified gain, certified X-failure, HNShape, BudgetShape, graph-core descent, or a governed UnaryWindow.*

## 7.4 Unary decoder UN

A unary path is

$$P = (v_0, e_1, v_1, \dots, e_m, v_m).$$

The finite state alphabet is

$$\Sigma_{UN} = \Sigma_{car} \times \Sigma_{pref} \times \Sigma_{orig} \times \Sigma_{ker} \times \Sigma_{obl} \times \Sigma_{dir} \times \Sigma_{sat} \times \Sigma_{bud} \times \Sigma_{act} \times \Sigma_{chg}.$$

Each edge has full and projected transition relations  $T_i^1, T_i^0$ . Forward and backward full sets are

$$\begin{aligned} F_0 &= B_0, \text{quad} F_i = T_i^1(F_{i-1}), \\ R_m &= B_m, \quad R_{i-1} = (T_i^1)^{-1}(R_i). \end{aligned}$$

A full lift exists iff

$$F_m \cap B_m \neq \emptyset.$$

If a projected lift exists but no full lift exists, UN outputs a canonical minimal blocked interval  $[a, b]$  with

$$T_{[a,b]}^1(F_a) \cap R_b = \emptyset,$$

and

$$T_{[a,b]}^0(\rho F_a) \cap \rho R_b \neq \emptyset.$$

**Theorem 7.3** (Package UN). *Every governed connected full-span unary window routes to a  $\text{VerifyDW}_{\text{exp}}$ -accepted gain, certified X-failure, HNShape, BudgetShape, SelectorSeed, BCLeak, strict unary descent, or clean full-span HSC-spine token.*

BC and UN never import each other. BC emits ‘UnaryWindow’ and UN emits ‘BCLeak’ as *EmitToken* route objects only. Route scheduling consumes such tokens later.

## 8 Hereditary normal forms, HResolve, and BUD

### 8.1 Hereditary package HN

Hereditary leaves have type

$$\tau \in \{\text{pair}, \text{tripod}, \text{spine}, \text{nf}\},$$

corresponding to HSC-pair, HSC-tripod, HSC-spine, and hereditary non-flatness. A certificate is

$$L = (\tau, U, \Gamma, \text{shape}, \text{blocks}, \text{frontier}, \text{charge}, \Pi_{\text{struct}}).$$

The block decomposition is

$$U = B_1 \cup \dots \cup B_m.$$

The BWL layered graph minimizes

$$(\text{cost}, \text{resrank}, \text{frdev}, \text{dwcode}).$$

**Theorem 8.1** (BWL core exactness). *The BWL shortest path computes the exact minimum inside the certified hereditary grammar and reconstructs a frontier-faithful realization.*

Each hereditary type has local normal rules LN1 to LN10. Critical pairs are generated over bounded overlaps. Each bounded peak is joined or emits VerifyDWGain, BudgetShape, HNShape, SelectorSeed, LeafDescent, or a finite critical row. Termination plus critical-pair coverage gives confluence on no-exit components.

ParseOrExit completeness states that every same-frontier equivalent open word either parses into the hereditary grammar after no-exit normalization, or activates a high-priority exit.

**Theorem 8.2** (Leaf-tightness). *If  $L$  is accepted and no HN high-priority exit is active, the BWL output  $R_L$  satisfies*

$$[[R_L]] = F_{C,U}, \quad \mathcal{F}(R_L) = \mathcal{F}(U), \quad |R_L| = \mu^*(F_{C,U}).$$

### 8.2 Global hereditary resolver

HResolve enumerates accepted proper hereditary leaves and full-span side-closed HSC-spines. Full-span spines are solved first. Proper leaves are assembled into a maximal H-disjoint family. H-disjointness includes support, frontier, origin, kernel, obligation, prefix-tail, charge, and interface noninterference.

The NoHereditary sidecar is strong and blocker-indexed. Each hereditary candidate has an entry H0 through H4: invalid, solved, budget-blocked, selector-blocked, or neutral/dead.

**Theorem 8.3** (HResolve). *The global hereditary resolver has three sound outputs:*

$$\begin{aligned} HResolve_{\text{exp}}(C) &= \text{Gain}(U, R, \Pi) \Rightarrow \text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept}, \\ HResolve_{\text{exp}}(C) &= \text{Minimum}(C, M, \mathcal{M}) \Rightarrow [[M]] = [[C]] \text{ and } |M| = \mu(C), \\ HResolve_{\text{exp}}(C) &= \text{NoHereditary}(C, \mathcal{N}_H), \end{aligned}$$

where  $\mathcal{N}_H$  is a total sidecar over governed hereditary candidates.

### 8.3 Budget resolver

Each budget row has a budget vector

$$\text{bud}(r) = (M_0(r), B_1(r)) \in \mathbb{Z}_{\geq 0}^2.$$

For a saturated hull  $H$ ,

$$M_0(H) = \sum_{r \in H} M_0(r), \quad B_1(H) = \sum_{r \in H} B_1(r).$$

The bounds  $M_0^{\text{max}}(H), B_1^{\text{max}}(H)$  are certified upper envelopes over a finite neutral no-outcome grammar, computed by dynamic programming.

BudgetResolve routes active certificates through proper, disconnected full-span, branch-cycle, unary, full-spine, HN, packet, exact-route, or descent routes. It emits a strong NoBudget sidecar. Each budget candidate has entry B0 through B4: invalid, within envelope, hereditary-blocked, selector-blocked, or solved/routed.

**Theorem 8.4** (BudgetResolve). *BudgetResolve has three sound outputs:*

$$\begin{aligned} \text{BudgetResolve}_{\text{exp}}(C) = \text{Gain}(U, R, \Pi) &\Rightarrow \text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept}, \\ \text{BudgetResolve}_{\text{exp}}(C) = \text{Minimum}(C, M, \mathcal{M}) &\Rightarrow [[M]] = [[C]], \quad |M| = \mu(C), \\ \text{BudgetResolve}_{\text{exp}}(C) = \text{NoBudget}(C, \mathcal{N}_B), & \end{aligned}$$

where  $\mathcal{N}_B$  is a total sidecar over governed budget candidates.

HN and BUD may reference each other only through sidecar blocker edges. Actual simultaneous acyclicity is checked later by HB.

## 9 Frontier-faithful comparison and strong neutral overlay

### 9.1 Strong neutral overlay

A saturated hull has full frontier

$$\mathcal{F}_H = (\partial_H, \iota_H, \text{car}_H, \text{orig}_H, \text{ker}_H, \text{obl}_H, \text{pref}_H, \text{dir}_H, \text{sat}_H, \text{bud}_H, \text{chg}_H).$$

A replacement is frontier-faithful if it preserves this ordered frontier under explicit transport.

A strong neutral overlay between same-frontier full-mode open direct-wire words  $H$  and  $R$  is a bijection

$$M : \text{Ch}(H) \rightarrow \text{Ch}(R)$$

that preserves weights, type, truth vectors, relation tables, carrier, origin, kernel, obligation, prefix, direction, saturation, budget, charge records, ordered frontier, dependency edges, source-port incidence, and consumer-port incidence.

**Theorem 9.1** (Strong neutral overlay rigidity). *If  $(H, R)$  admits a strong neutral overlay, then*

$$|H| = |R|.$$

Moreover,  $H$  and  $R$  are direct-wire congruent up to finite carrier/profile transport.

### 9.2 FF-LOSS and frontier-faithful comparison

Let

$$R^{(0)} \rightarrow R^{(1)} \rightarrow \dots \rightarrow R^{(T)}$$

be the frontier-closure sequence for a cheaper comparison word. A first-loss atom is the first added frontier materializer whose charge destroys strict saving relative to  $H$ . Its minimal loss cone is

$$\mathcal{K}(a) = \text{Comp}(\text{Sat}(\text{Dep}^-(a) \cup \text{Witness}(a))).$$

**Theorem 9.2** (FF-LOSS). *Every first saving-destroying frontier closure atom routes to one of*

$$\text{CritC}, Q, E, L, X1, X2, X3, X4,$$

or to a verified gain, BC, UN, HN, BUD, selector, exact route, or strict descent. If the loss cone is neutral-exact, it routes to strict residual-rank descent or exact route.

**Theorem 9.3** (Frontier-faithful comparison reduction). *In a no-outcome branch, every full-positive witness has a cheaper same-frontier, frontier-faithful full-mode comparison word. If frontier closure fails or destroys saving, FF-LOSS routes the first failed or first-loss coordinate.*

## 10 Residual witness normal form

### 10.1 Terminal whole-carrier bridge

For the whole circuit  $C$ , define the terminal full-carrier function

$$F_C := F_C^{top,1}.$$

This terminal carrier preserves global Boolean semantics and full-mode admissibility, but it does not force all internal origin, kernel, prefix, obligation, budget, charge, and proof records of  $C$  to be externally preserved semantic data.

**Theorem 10.1** (RW-MuBridge, terminal form). *For every normalized whole circuit  $C$ ,*

$$\mu(C) = \mu_1^*(F_C^{top,1}).$$

There are size-preserving maps: terminalization of a whole circuit into a closed full word and realization of a closed full word as a whole circuit. These maps prove the two inequalities.

### 10.2 Full-span policy

A full-span witness  $H = C$  is legal as a residual witness, but it cannot be emitted as a local  $\text{VerifyDW}_{\text{exp}}$  gain. A cheaper whole-span realization gives strict residual descent; an exact whole-span realization gives exact route.

**Lemma 10.2** (Whole-span cheaper word gives descent). *If  $R$  is a closed full word with  $[[R]] = F_C$  and  $|R| < |C|$ , then  $C' = \text{Real}_C(R)$  satisfies  $[[C']] = [[C]]$  and*

$$\Lambda(C') < \Lambda(C).$$

### 10.3 SaturatePositive and RankWF

A residual witness candidate  $H^0$  may not be saturated. Run deterministic saturation and completion:

$$H^0 \rightarrow H^1 \rightarrow \dots \rightarrow H^T = \text{Comp}(\text{Sat}_C(H^0)).$$

Let

$$\Phi_t = |H^t| - \mu_1^*(F_{H^t}^1), \quad D_t = \mu_1^*(F_{H^t}^1) - \mu_0^*(F_{H^t}^0).$$

Transparent saturation steps add the same forced full-mode cost to the support and to the minimum, and add no more quotient cost than full cost. Thus full positivity is preserved and projection positivity does not decrease.

**Theorem 10.3** (RW-SaturatePositive). *Saturating a positive residual witness either preserves full or projection positivity, or routes the first positivity-loss coordinate to CritC, Q, E, L, X, gain, BC, UN, HN, BUD, selector, exact route, or strict descent.*

The dangerous case is interface exposure: adding a required outgoing coordinate can increase semantic obligation without adding a physical gate. This must route to  $E$  unless the exposure is a zero-cost retract.

Residual ranks are finite lexicographic tuples over witness type, span type, mode, frontier defect, projection defect, saturation defect, anchor count, charge size, profile size, and canonical code. Therefore the rank order is well-founded.

### 10.4 BCEL-ready residual witnesses

A BCEL-ready record is

$$\text{BCELReady}(C, H, A, \mathcal{X}, D).$$

It certifies:

1.  $H$  is governed, compatible, saturated, completed, and projection-positive:

$$d(H) = D > 0.$$

2. A finite anchor set  $A = A(H)$  partitions primitive records.
3. For every  $S \subseteq A$ ,

$$X_S = \text{Comp}(\text{Sat}_C(\bigcup_{a \in S} P_a)).$$

4. The anchor family is Boolean:

$$X_S \wedge X_T = X_{S \cap T}, \quad X_S \vee X_T = X_{S \cup T}.$$

5. All anchor squares are projection-compatible.
6.  $A$  is a minimal positive nucleus:

$$d(X_A) = D > 0, \quad B \subsetneq A \Rightarrow d(X_B) = 0.$$

7.  $|A| \geq 2$ .
8. For every nonempty proper cut,

$$\Omega(S, A \setminus S) = D.$$

**Theorem 10.4** (RW-BCELReady). *If  $C$  is normalized,  $\Lambda(C) > 0$ , and no verified gain, exact route, named outcome, selector route, or strict residual descent is active, then there exists a BCEL-ready projection-positive hull.*

The Boolean anchor algebra is not automatic. The checker must audit non-Boolean saturation triggers and route them to a named outcome, selector, exact route, gain, or strict descent. In no-outcome branches, no such trigger remains.

## 11 BN2 through BN6 and Package C

### 11.1 BN2: side-tight coherent optima

For a legal saturated projection-compatible square

$$\begin{array}{ccc} I & \longrightarrow & A \\ \downarrow & & \downarrow \\ B & \longrightarrow & U, \end{array}$$

define

$$\delta_i(A, B) = m_i(A) + m_i(B) - m_i(I) - m_i(U).$$

A square-coherent tuple is

$$\mathcal{X} = (X_I, X_A, X_B, X_U, \lambda_A, \lambda_B, \lambda_U, \Theta).$$

Its slacks are

$$\epsilon_S = |X_S| - m_i(S).$$

Its incidence value is

$$v_{\mathcal{X}} = |X_A| + |X_B| - |X_I| - |X_U|.$$

Therefore

$$v_{\mathcal{X}} = \delta_i(A, B) + \epsilon_A + \epsilon_B - \epsilon_I - \epsilon_U.$$

A coherent tuple is side-tight iff every  $\epsilon_S = 0$ .

**Theorem 11.1** (BN2-CoherentOptimum). *In a no-outcome branch, every legal saturated projection-compatible square has a side-tight square-coherent tuple in each mode  $i$ . For every side-tight basis,*

$$v_G = \delta_i(A, B).$$

Consequently,

$$\delta_i(A, B) = \max_{G \in \mathcal{G}_i^{\text{tight}}(A, B)} v_G.$$

BN2 does not allow no-overclaim for arbitrary coherent bases. It is valid only for side-tight bases.

## 11.2 BN3: simultaneous finite request envelopes

For each mode  $i$ , BN3 constructs a finite weighted request system

$$(P_i, \leq_i, \nu_i, D_i).$$

A unit  $p \in P_i$  has a stable request predicate

$$r_p^{(i)}(T) = 1_{p \in D_i(T)}.$$

It is represented by a minimal-consumer antichain

$$\mathcal{M}_p^{(i)} = \min_{\subseteq} \{T \subseteq A : T \neq \emptyset, r_p^{(i)}(T) = 1\}.$$

Then

$$r_p^{(i)}(T) = 1 \iff \exists M \in \mathcal{M}_p^{(i)} [M \subseteq T].$$

**Theorem 11.2** (BN3-SimultaneousEnvelope). *For each mode  $i$ , there exists a finite weighted request system such that for every proper cut  $S$ ,*

$$\delta_i^{\text{cut}}(S, A \setminus S) = \sum_{p \in P_i} \nu_i(p) r_p^{(i)}(S) r_p^{(i)}(A \setminus S).$$

*The request predicates are stable, monotone, vanish on the empty set, and jointly realizable in one side-tight BN2 basis.*

Raw individual requestability is insufficient. BN3 requires stable requestability plus joint realizability.

## 11.3 BN4: activation-exact cancellation

Define the active consumer antichain

$$\mathcal{A}_p^{(i)} = \{M \in \mathcal{M}_p^{(i)} : \exists N \in \mathcal{M}_p^{(i)}, M \cap N = \emptyset\}.$$

If  $\mathcal{A}_p = \emptyset$ , then  $p$  is cut-silent. The activation is

$$\alpha_p(S) = r_p(S) r_p(A \setminus S).$$

Activation equality is certified by active-antichain equality:

$$\alpha_p = \alpha_q \iff \mathcal{A}_p = \mathcal{A}_q.$$

No package may certify activation equality by enumerating all cuts.

A BN4 key is

$$\kappa(p) = (\bar{\sigma}(p), \text{ActCode}(p), \tau(p)),$$

where  $\bar{\sigma}$  is the projected semantic signature and  $\tau$  is projected transport type.

**Theorem 11.3** (BN4-ActivationExact). *For every proper cut,*

$$\Omega(S, A \setminus S) = \sum_{c_\kappa > 0} c_\kappa \alpha_\kappa(S) - \sum_{c_\kappa < 0} (-c_\kappa) \alpha_\kappa(S).$$

*After cancellation, no positive and negative residual cells share the same activation-exact key.*

## 11.4 BN5: full-shadow localization

For a negative full residual key  $\kappa$ , refine masses into unit atoms and form a shadow graph

$$\mathcal{B}_\kappa = (L_\kappa, R_\kappa, E_\kappa),$$

where left vertices are full atoms and right vertices are quotient candidate shadows with the same activation-exact key. Hall deficits and first unmatched full atoms localize to CritC rows or  $X$ -failures.

**Theorem 11.4** (BN5-FullShadowLocalization). *Every cut-active negative full residual key routes to*

$$\text{CritC}, Q, E, L, X1, X2, X3, X4,$$

*or to verified gain, BC, UN, HN, BUD, selector, exact route, or strict descent. Therefore, in a no-outcome branch, every negative full residual key is cut-silent.*

Thus on proper cuts,

$$\Omega(S, A \setminus S) = \sum_{\kappa \in Q^{\text{res}}} \gamma(\kappa) \alpha_\kappa(S) \geq 0.$$

## 11.5 Package C, renamed PkgC: separating consumers

The package formerly denoted Package C is named PkgC to avoid ambiguity with the critical kind CritC.

For a surviving quotient residual key  $\kappa$ , if its active minimal-consumer antichain has a disjoint pair  $(M, N)$  with at least one nonsingleton member, then PkgC builds request traces, first multi-active joins, activation-exact full-restoration universes, and quotient-to-full matching graphs. Either the quotient residual mass is fully matched, contradicting BN4 residuality, or Hall failure yields a kind  $Q$  row, an  $X$ -failure, or an unbounded route.

**Theorem 11.5** (Separating-consumer theorem). *In a minimum-rank no-outcome branch, every disjoint active pair of minimal consumers for a surviving quotient residual key consists of singleton sets.*

Therefore V54 applies. For each surviving active quotient residual key, there exists

$$E(\kappa) = \{a : \{a\} \in \mathcal{M}_\kappa\}, \quad |E(\kappa)| \geq 2,$$

with

$$\alpha_\kappa(S) = 1_{E(\kappa) \cap S}.$$

## 11.6 BN6: hypergraph cellization and packet collapse

Group equal footprints:

$$w_E = \sum_{\kappa: E(\kappa)=E} \gamma(\kappa).$$

Then

$$\Omega(S, A \setminus S) = \sum_{\substack{E \subseteq A \\ |E| \geq 2}} w_E 1_{E \cap S}, \quad w_E \in \mathbb{Z}_{\geq 0}.$$

Together with the constant-cut equation

$$\Omega(S, A \setminus S) = D > 0,$$

V53 yields packet prototypes.

**Theorem 11.6** (BN6-HypergraphPacket). *Every BCEL-ready positive nucleus in a minimum-rank no-outcome branch produces a positive packet prototype of type*

$$\text{Pair}, \quad \text{BalancedTri}, \quad \text{FullSpan}.$$

*Every positive packet mass carries atom witnesses.*

For  $|A| = 3$ , the case can be mixed:

$$D = 2p + w_A.$$

Both a balanced-triple packet and a full-span packet may be present.

## 12 Finite rigidity and consumer antichains

**Theorem 12.1** (V53 constant-cut hypergraph rigidity). *Let  $A$  be finite with  $|A| = q \geq 2$ . Let  $w_E \in \mathbb{Z}_{\geq 0}$  for every  $E \subseteq A$ ,  $|E| \geq 2$ . Assume*

$$\sum_{E \cap S} w_E = D > 0$$

*for every nonempty proper  $S \subsetneq A$ . Then:*

1. *if  $q = 2$ , then  $w_A = D$ ;*
2. *if  $q = 3$ , all pair weights are equal, say  $p$ , and  $w_A = D - 2p$ ;*
3. *if  $q \geq 4$ , every proper hyperedge weight vanishes and  $w_A = D$ .*

*Proof.* For  $T \subseteq A$ , set

$$W(T) = \sum_{E \subseteq T, |E| \geq 2} w_E, \quad W = W(A).$$

A hyperedge crosses  $S$  exactly when it is not contained in  $S$  and not contained in  $A \setminus S$ . Hence

$$\sum_{E \cap S} w_E = W - W(S) - W(A \setminus S) = D,$$

so

$$W(S) + W(A \setminus S) = K := W - D.$$

For singleton  $\{i\}$ ,  $W(A \setminus \{i\}) = K$ . For pair  $\{i, j\}$ ,

$$w_{ij} + W(A \setminus \{i, j\}) = K.$$

Subtracting gives

$$w_{ij} = \sum_{\substack{E \subseteq A \setminus \{i\} \\ j \in E, |E| \geq 2}} w_E.$$

For  $q = 3$ , this forces all pair weights equal. For  $q \geq 4$ , the right side contains at least two distinct pair terms of the same common weight, forcing that common pair weight to be zero. Then every proper hyperedge lies in such a zero sum and vanishes. The cases follow.  $\square$

**Theorem 12.2** (V54 consumer-antichain normal form). *Let  $r : 2^A \rightarrow \{0, 1\}$  be monotone with  $r(\emptyset) = 0$ . Let*

$$\mathcal{M} = \min_{\subseteq} \{S \neq \emptyset : r(S) = 1\}.$$

*Define*

$$\kappa(S) = r(S)r(A \setminus S).$$

*Then  $\kappa \not\equiv 0$  iff  $\mathcal{M}$  contains two disjoint members. If every disjoint pair in  $\mathcal{M}$  consists of singleton sets, then  $\kappa$  is exactly the cut indicator of*

$$E = \{a : \{a\} \in \mathcal{M}\}.$$

## 13 Packet extraction and selector coverage

### 13.1 Selector universe

The final selector universe is

$$\boxed{\mathcal{K}(C) = K_2(C) \cup K_3(C) \cup K_{sp}(C).}$$

The pair selector family is

$$K_2(C) = \{(pair, h_1, h_2, \theta_2) : h_1, h_2 \in H(C), \theta_2 \in \Theta_2(|C|)\}.$$

The triple selector family is

$$K_3(C) = \{(tri, h_1, h_2, h_3, \theta_3) : h_i \in H(C), \theta_3 \in \Theta_3(|C|)\}.$$

The spine selector family is

$$K_{sp}(C) = \{(sp, h_{in}, h_{cut}, h_{out}, \theta_{sp}) : h_* \in H(C), \theta_{sp} \in \Theta_{sp}(|C|)\}.$$

The handle bound and payload bounds give

$$|\mathcal{K}(C)| \leq |C|^3 (\log |C|)^{O(1)}.$$

### 13.2 Pair packet extraction

A pair packet

$$Pair(a, b; M, Cell(\{a, b\}))$$

with  $M > 0$  has an atom witness  $u \in Cell(\{a, b\})$ . Its payload is

$$\theta_2(u) = (\bar{\sigma}(u), \tau(u), \mathcal{F}(u), \text{Ch}(u), \text{Obl}(u), \text{Orig}(u), \text{Ker}(u), \text{Pref}(u), \text{Dir}(u), \text{Bud}(u), \text{ActionCode}(u), \text{ranktag}(u)).$$

Then

$$K_u = (pair, h(a), h(b), \theta_2(u)) \in K_2(C).$$

**Theorem 13.1** (Pair packet seed). *Every positive pair packet yields a faithful  $K_2$  selector seed, or routes a first colour, frontier, charge, obligation, activation, direction, budget, rank, exact-route, or descent obstruction.*

### 13.3 Balanced-triple extraction

A balanced triple gives nonempty equal-mass cells

$$Cell(ab), \quad Cell(ac), \quad Cell(bc).$$

A compatible triad

$$(u_{ab}, u_{ac}, u_{bc})$$

must satisfy endpoint consistency, projected signature coherence, frontier coherence, charge coherence, obligation coherence, direction coherence, budget coherence, activation coherence, and rank coherence.

**Theorem 13.2** (Balanced-triple seed). *Every positive balanced-triple packet yields a faithful  $K_3$  selector seed, or routes the first triad mismatch. A single compatible triad is enough; a perfect matching of all triple mass is not required.*

### 13.4 Full-span spine extraction

A full-span packet

$$FullSpan(A; M, Cell(A))$$

with  $M > 0$  has atom witnesses with footprint  $A$ . For a least witness  $u$ , form the trace support

$$\mathcal{S}(u) = \text{Comp}(\text{Sat}_C(\text{Dep}^-(u) \cup \bigcup_{a \in A} \text{ReqTr}(u, a))).$$

Its primitive incidence graph  $\Gamma(u)$  must either be a clean spine or route.

A clean full-span spine is connected, path-like after suppressing neutral degree-two profile-only vertices, covers exactly the full-span footprint, has accepted unary transitions, frontier-faithful prefixes, full-mode obligation cleanliness, unique charge ownership, budget within envelope, no unresolved hereditary side closure, no lower packet, and proper decoded support.

It compresses into three handles:

$$(h_{in}, h_{cut}, h_{out})$$

and a finite payload  $\theta_{sp}$ . The deterministic spine walker reconstructs the path inside  $C$ .

**Theorem 13.3** (Full-span spine seed). *Every positive full-span packet yields a faithful  $K_{sp}$  selector seed, or routes to CritC,  $Q, E, L, X$ , verified gain, BC, UN, HN, BUD, selector descent, exact route, or strict residual descent.*

**Theorem 13.4** (BCEL seed, patched). *If  $C$  is normalized,  $\Lambda(C) > 0$ , and no verified gain, exact route, named outcome, selector descent, strict residual descent, HN-active object, or budget-active object survives, then*

$$\exists K \in K_2(C) \cup K_3(C) \cup K_{sp}(C), \quad \exists P, \quad \text{Faithful}(C, K, P).$$

## 14 Package R: selector realization and strict surplus

### 14.1 Public realizer contract

For every selector

$$K \in K_2(C) \cup K_3(C) \cup K_{sp}(C),$$

the public realizer output type is

$$\text{Real}(C, K) \in \{\perp, \text{Gain}(U, R, \Pi)\}.$$

A gain is returned only after

$$\text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept}.$$

A  $\perp$ -certificate for a faithful selector is one of:

$$BotHN, \quad BotBUD, \quad BotSeed.$$

The static reject case is impossible for faithful selectors.

**Theorem 14.1** (Selector realization). *For every selector  $K$ ,*

$$\text{Real}(C, K) = \text{Gain}(U, R, \Pi) \implies \text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept}.$$

*Moreover, if*

$$\text{Faithful}(C, K, P)$$

*and*

$$\text{Real}(C, K) = \perp,$$

*then*

$$\text{HNActive}_{\leq P}(C) \vee \text{BudgetActive}_{\leq P}(C) \vee \exists K', P' [\text{Faithful}(C, K', P') \wedge P' \prec P].$$

## 14.2 Strict charge surplus

A selector blueprint has a charge-surplus injection if there is an injective map

$$\psi_K : \text{Ch}(R_K) \hookrightarrow \text{Ch}(U_K)$$

preserving weights, type, truth vectors, relation tables, frontier, origin, kernel, obligation, prefix, direction, saturation, budget, charge, source incidence, consumer incidence, and dependency edges, with at least one positive-weight unmatched support charge.

**Theorem 14.2** (R-ChargeSurplus). *If*

$$\text{Faithful}(C, K, P), \quad \text{NoHN}_{\leq P}(C), \quad \text{NoBUD}_{\leq P}(C), \quad \text{NoFaithful}_{< P}(C),$$

*then the decoded selector blueprint admits a charge-surplus injection. Consequently,*

$$|R_K| \leq |U_K| - 1,$$

*and a  $\text{VerifyDW}_{\text{exp}}$ -accepted gain is produced.*

Pair selectors fold two active traces into one, leaving one residual atom unmatched. Balanced triples fold three compatible edge traces into two selector-core traces. Spines compress a clean path and omit at least one interior transition charge unless HN-active, budget-active, or lower-rank packet structure is present.

## 15 Simultaneous HN-BUD rank-negative closure

Let  $(\mathcal{P}_C, \prec)$  be the finite packet-rank set. A rank has the form

$$P = (\text{kind}, \text{span}, \text{massclass}, \text{colourclass}, \text{frontierclass}, \text{chargeclass}, \\ \text{budgetclass}, \text{hereditaryclass}, \text{activationclass}, \text{ranktag}, \text{code}).$$

Define

$$\text{NoFaithful}_{< P}(C) := \neg \exists K, Q [K \in \mathcal{K}(C) \wedge Q \prec P \wedge \text{Faithful}(C, K, Q)].$$

A hereditary active certificate  $L$  has rank  $\rho_H(L)$ . A budget active certificate  $B$  has rank  $\rho_B(B)$ . The combined blocker graph has H vertices and B vertices. H2 sidecar entries give edges

$$H(L) \rightarrow B(B),$$

and B2 entries give edges

$$B(B) \rightarrow H(L).$$

Every edge satisfies

$$\rho(\text{dst}) \leq \rho(\text{src}),$$

and if ranks are equal,

$$\beta(\text{dst}) < \beta(\text{src}).$$

Thus the graph is acyclic.

**Theorem 15.1** (HB negative closure). *For every packet rank  $P$ ,*

$$\text{NoHereditary}(C, \mathcal{N}_H) \wedge \text{NoBudget}(C, \mathcal{N}_B) \wedge \text{NoFaithful}_{< P}(C) \implies \text{NH}_{\leq P}(C) \wedge \text{NB}_{\leq P}(C).$$

*Proof.* Assume an active HN or BUD vertex of rank at most  $P$ , and choose one minimizing  $(\rho, \beta)$ . The local sidecar soundness rule forces a blocker edge to an active vertex of strictly smaller  $(\rho, \beta)$ , contradiction.  $\square$

**Theorem 15.2** (Selector-silence induction). *If selector silence, NoHereditary, and NoBudget sidecars are recorded, and no lower branch is active, then for every rank  $P$ ,*

$$\text{NoFaithful}_{\leq P}(C), \quad \text{NH}_{\leq P}(C) \wedge \text{NB}_{\leq P}(C).$$

## 16 Oracle, ZeroSlack, and residual-band minimization

### 16.1 Rank-ordered PCCOracle

The oracle enumerates selectors in packet-rank order.

```
PCCOracle_exp(C):
  h = HResolve_exp(C)
  if h is Minimum(C,M,Mc): return Minimum(C,M,Mc)
  if h is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
  store NoHereditary(C,N_H)

  b = BudgetResolve_exp(C)
  if b is Minimum(C,M,Mc): return Minimum(C,M,Mc)
  if b is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
  store NoBudget(C,N_B)

  construct finite rank list P_1,...,P_M
  for j = 1 to M:
    for K in K_{P_j}(C):
      out = Real(C,K)
      if out is Gain(C,U,R,Pi): return Gain(C,U,R,Pi)
      store Real(C,K)=bot
    store SelectorSilent_{<=P_j}(C)

  return ZeroSlack(C,Z)
```

The ZeroSlack certificate stores normalized-input record, NoHereditary sidecar, NoBudget sidecar, HB blocker graph, rank list, selector silence ledger for  $K_2, K_3, K_{sp}$ , no-lower ledger, no-faithful rank induction,  $\text{NH}_{\leq P} \wedge \text{NB}_{\leq P}$  rank induction, and BCEL contradiction proof.

**Theorem 16.1** (Rank-parametric ZeroSlack). *If*

$$\text{PCCOracle}_{\text{exp}}(C) = \text{ZeroSlack}(C, \mathcal{Z}),$$

*then*

$$\Lambda(C) = 0.$$

*Proof.* Assume  $\Lambda(C) > 0$ . The residual witness and BCEL stack yield an allowed earlier route or a faithful selector. The no-lower ledger excludes earlier routes. The HB rank induction excludes HN and budget active blockers and proves no faithful selector exists at any rank under selector silence. Contradiction.  $\square$

### 16.2 PCCMin

```
PCCMin_exp(C0):
  C = C0
  while True:
    n = NormalizeOrGain_exp(C)
    if n is Gain(C,U,R,Pi):
      C = C[U <- R]
      continue
    if n is Normal(Cnu):
      C = Cnu
```

```

out = PCCOracle_exp(C)
if out is Minimum(C,M,Mc): return M
if out is ZeroSlack(C,Z): return C
if out is Gain(C,U,R,Pi):
    C = C[U <- R]

```

Every nonterminal gain step lowers  $\Lambda$  by at least one. Hence the number of gain steps is at most  $\Lambda(C_0)$ .

**Theorem 16.2** (Residual-band exact minimization). *If the accepted package is valid and*

$$\Lambda(C_0) \leq O(\log |C_0|),$$

*then  $\text{PCCMin}_{\text{exp}}(C_0)$  returns an exact minimum equivalent circuit in polynomial time.*

## 17 Locked NAND SAT embedding

### 17.1 Carrier slots and invariants

The locked NAND builder uses tagged disjoint carrier families

$$X \sqcup T \sqcup O \sqcup R \sqcup L \sqcup \{z\}.$$

Here  $X$  are primary inputs,  $T = \{t_j\}$  are trace value slots,  $O = \{o_e\}$  are source occurrence slots,  $R = \{r_e\}$  are source locks,  $L = \{\ell_j\}$  are trace locks, and  $z$  is the final fresh lock.

The invariant  $G\text{-Sep}^+$  requires pairwise distinct inputs inside equality and trace macros, fresh occurrence slots for duplicate NAND sources, constants enforced by  $M_0, M_1$  rather than carrier constants, globally fresh macro locks, final lock  $z$  appearing only in the final output construction, and prefix-conjunction nodes with distinct underlying check-lock sets.

The invariant  $G\text{-Coh}$  gives a coherence witness assigning all distinguished checks the value 1.

### 17.2 Macros

The equality macro  $M_{=}$  on inputs  $(r, u, s)$  has ten gates:

$$\begin{aligned} a_1 &= \mathbf{N}(r, u), & a_2 &= \mathbf{N}(r, s), & a_3 &= \mathbf{N}(a_1, a_1), & a_4 &= \mathbf{N}(a_3, s), \\ a_5 &= \mathbf{N}(r, a_1), & a_6 &= \mathbf{N}(a_5, a_5), & a_7 &= \mathbf{N}(a_6, a_2), & a_8 &= \mathbf{N}(a_4, a_7), \\ a_9 &= \mathbf{N}(r, r), & a_{10} &= \mathbf{N}(a_9, u). \end{aligned}$$

The distinguished output is

$$a_8 = r[(u \wedge s) \vee (\neg u \wedge \neg s)].$$

All ten outputs are exposed.

The constant-one macro is

$$b_1 = \mathbf{N}(r, u), \quad b_2 = \mathbf{N}(b_1, b_1) = r \wedge u.$$

The constant-zero macro is

$$d_1 = \mathbf{N}(r, u), \quad d_2 = \mathbf{N}(r, d_1), \quad d_3 = \mathbf{N}(d_2, d_2) = r \wedge \neg u.$$

The NAND trace macro  $M_N$  on inputs  $(\ell, t, u, v)$  has eighteen gates:

$$\begin{aligned} q_1 &= \mathbf{N}(\ell, t), & q_2 &= \mathbf{N}(q_1, q_1), & q_3 &= \mathbf{N}(\ell, u), & q_4 &= \mathbf{N}(\ell, v), \\ q_5 &= \mathbf{N}(q_2, q_3), & q_6 &= \mathbf{N}(q_2, u), & q_7 &= \mathbf{N}(q_6, q_6), & q_8 &= \mathbf{N}(q_7, q_4), \\ q_9 &= \mathbf{N}(\ell, q_1), & q_{10} &= \mathbf{N}(q_9, q_9), & q_{11} &= \mathbf{N}(q_{10}, u), & q_{12} &= \mathbf{N}(q_{11}, q_{11}), \\ q_{13} &= \mathbf{N}(q_{12}, v), & q_{14} &= \mathbf{N}(q_5, q_8), & q_{15} &= \mathbf{N}(q_{14}, q_{14}), & q_{16} &= \mathbf{N}(q_{15}, q_{13}), \\ q_{17} &= \mathbf{N}(\ell, \ell), & q_{18} &= \mathbf{N}(q_{17}, t). \end{aligned}$$

The distinguished output is

$$q_{16} = \ell[t = \mathbf{N}(u, v)].$$

All eighteen outputs are exposed.

Prefix conjunction of two check bits uses two gates and exposes  $p$  and  $\neg p$ . The final ternary conjunction uses four gates and exposes only

$$F_\varphi = z \wedge T_\varphi \wedge y_{out},$$

where  $z$  is a fresh final lock.

### 17.3 Baseline and threshold

For a NAND circuit  $\varphi$  with  $m$  gates and source occurrence counts  $w_=, w_0, w_1$ , the baseline is

$$B_\varphi^{NAND} = 18m + 10w_= + 3w_0 + 2w_1 + 2(3m - 1).$$

The full word has

$$|W_\varphi^{NAND}| = B_\varphi^{NAND} + 4.$$

**Theorem 17.1** (BaselineDistinct). *The baseline tuple contains exactly  $B_\varphi^{NAND}$  pairwise distinct nonconstant nonprojection functions. Hence every direct-wire realization of the baseline tuple has at least  $B_\varphi^{NAND}$  NAND gates, and the constructed baseline is exact.*

**Theorem 17.2** (Locked NAND threshold). *For every NAND circuit  $\varphi$ ,*

$$\varphi \notin SAT \implies \mu(W_\varphi^{NAND}) = B_\varphi^{NAND},$$

$$\varphi \in SAT \implies B_\varphi^{NAND} + 1 \leq \mu(W_\varphi^{NAND}) \leq B_\varphi^{NAND} + 4.$$

Consequently,

$$\varphi \in SAT \iff \mu(W_\varphi^{NAND}) > B_\varphi^{NAND}, \quad \Lambda(W_\varphi^{NAND}) \leq 4.$$

### 17.4 Output convention and threshold proof details

The locked NAND threshold uses the direct-wire multi-output convention of Section 2.1. An output coordinate may be wired to a boundary coordinate, to a carrier constant, or to the output of a NAND gate. Repeating an already available output coordinate does not add a NAND gate. This convention concerns only the external output tuple. It does not make checked data constants free inside the locked construction: constants that participate in trace constraints are still enforced by the macros  $M_0$  and  $M_1$  as required by  $G\text{-Sep+}$ .

**Lemma 17.3** (DirectWireOutputLowerBound). *Let  $F = (f_1, \dots, f_B)$  be a direct-wire output tuple over a fixed carrier. If the functions  $f_i$  are pairwise distinct and each  $f_i$  is nonconstant and nonprojection, then every NAND direct-wire realization of  $F$  has at least  $B$  NAND gates.*

*Proof.* An output coordinate that is a boundary coordinate is a projection, and an output coordinate wired to a carrier constant is constant. Hence every nonconstant nonprojection output coordinate must be the output of some NAND gate. One NAND gate output computes one Boolean function. Since the  $f_i$  are pairwise distinct, no single gate output can serve two of them. Thus the realization contains at least  $B$  distinct NAND gate outputs, hence at least  $B$  NAND gates.  $\square$

**Lemma 17.4** (MacroDistinct). *Under  $G\text{-Sep+}$ , every exposed baseline output from an equality, constant, trace, or prefix macro is tagged by its macro instance and fresh lock or occurrence slot. The exposed baseline functions are pairwise distinct, nonconstant, and nonprojection. Therefore the baseline tuple forces exactly  $B_\varphi^{NAND}$  NAND gates.*

*Proof.* The accepted  $G$  certificate separates this lemma into the following explicit obligations: macro truth signatures are pairwise distinct, macro outputs are nonconstant, macro outputs are nonprojections, cross-instance copies are carrier-tagged, prefix outputs are separated from macro outputs, duplicate source occurrences use fresh slots, and the projection boundary is the positive boundary-coordinate convention of the direct-wire model.

For a single equality, constant, or NAND-trace macro instance, Appendix A gives the complete output truth signatures and the positive projection signatures in the declared variable order. Direct comparison of those finite bit strings shows that the exposed outputs of the instance are pairwise distinct and nonconstant. It also shows that no exposed output equals any positive projection of an input coordinate. This is the relevant projection test for the direct-wire convention: a boundary wire supplies a positive boundary coordinate, while

negations and other derived functions require NAND gates unless they are carrier constants or already available outputs.

Now consider two different macro instances. By  $G\text{-Sep+}$ , their private locks and occurrence variables are renamed into disjoint carrier coordinates. If two same-shape local signatures are copied into different instances, the resulting global functions depend on different carrier coordinates. Assign the first instance's active lock and vary the local data coordinate on which its output essentially depends while holding the second instance's carrier fixed; the first output changes and the second does not. Hence cross-instance copies are distinct global functions. Fresh occurrence slots for duplicate NAND sources are needed here: two occurrences of the same source value may be semantically equal on coherent traces, but they are distinct carrier functions before the trace constraints are imposed, so their exposed macro outputs do not collapse in the baseline tuple.

The prefix-conjunction outputs are separated for the same reason. Each prefix node uses its own check-lock set and depends essentially on a prefix check coordinate not used by any equality, constant, or NAND-trace macro output.  $G\text{-Sep+}$  gives distinct check-lock sets for different prefix nodes and keeps them disjoint from macro locks. Therefore a prefix output cannot equal a macro output or another prefix output unless the two tagged carrier supports are identical, which the construction forbids. The exposed negated prefix output  $\neg p$  is also not a free projection: it is a NAND-derived function over the prefix node's local carrier and is counted as an exposed baseline output.

Thus the whole baseline tuple contains exactly the displayed number of pairwise distinct nonconstant nonprojection functions:  $18m$  trace-macro outputs,  $10w_+$  equality outputs,  $3w_0$  constant-zero outputs,  $2w_1$  constant-one outputs, and  $2(3m - 1)$  prefix outputs. Applying  $\text{DirectWireOutputLowerBound}$  gives  $\mu(\text{baseline}) \geq B_\varphi^{\text{NAND}}$ . The constructed baseline exposes exactly those  $B_\varphi^{\text{NAND}}$  NAND outputs, so the lower bound is tight and the baseline is exact.  $\square$

**Lemma 17.5** (TraceEquivalence). *Let  $T_\varphi$  be the final prefix conjunction of all distinguished equality, constant, and NAND trace checks. For every assignment to the primary inputs  $X$ , there exists an extension to the trace, occurrence, and lock slots with  $T_\varphi = 1$  and  $y_{\text{out}} = 1$  if and only if  $\varphi(X) = 1$ . Consequently,  $T_\varphi \wedge y_{\text{out}}$  is satisfiable if and only if  $\varphi$  is satisfiable.*

*Proof.* The proof uses the following trace obligations, each of which is part of the accepted  $G$  certificate: active locks are forced by distinguished outputs; equality locks force equality of source and occurrence slots; constant locks force the occurrence value to be the stated constant; NAND trace locks force gate equations; the prefix covers all distinguished checks; the induction follows the topological order of the NAND circuit; the output slot  $y_{\text{out}}$  is the trace slot of the declared circuit output; and duplicate source occurrences are assigned fresh occurrence slots.

For the forward direction, suppose  $\varphi(X) = 1$ . Evaluate the NAND circuit in topological order. Assign  $t_j$  to the value of gate  $j$ , assign every source occurrence slot to the value of the source it names, and set every equality, constant, and trace lock to its active value. For an input or earlier-gate source, the corresponding  $M_+$  distinguished output is then

$$r[(u \wedge s) \vee (\neg u \wedge \neg s)] = 1,$$

because the occurrence value  $u$  and source value  $s$  agree. For a constant-one occurrence, the  $M_1$  distinguished output is  $r \wedge u = 1$ ; for a constant-zero occurrence, the  $M_0$  distinguished output is  $r \wedge \neg u = 1$ . For a NAND gate with source values  $u, v$  and trace value  $t$ , the  $M_N$  distinguished output is

$$\ell[t = \mathbf{N}(u, v)] = 1.$$

The prefix-conjunction tree is exact and includes each distinguished check once, so  $T_\varphi = 1$ . Since  $\varphi(X) = 1$ , the trace slot corresponding to the declared output gate has value 1, hence  $y_{\text{out}} = 1$ .

For the reverse direction, suppose there is an extension with  $T_\varphi = 1$  and  $y_{\text{out}} = 1$ . Exactness and coverage of the prefix tree imply that every distinguished check is one. For each equality check, the equation

$$r[(u \wedge s) \vee (\neg u \wedge \neg s)] = 1$$

forces  $r = 1$  and  $u = s$ . For constant checks,  $r \wedge u = 1$  forces  $u = 1$ , and  $r \wedge \neg u = 1$  forces  $u = 0$ . For each NAND trace check,

$$\ell[t = \mathbf{N}(u, v)] = 1$$

forces  $\ell = 1$  and  $t = \mathbf{N}(u, v)$ . Fresh occurrence slots ensure that duplicate source occurrences are separately constrained and cannot be silently identified. By induction over the topological order of the NAND DAG, every trace slot therefore equals the value obtained by evaluating the corresponding gate of  $\varphi$  on  $X$ : the base cases are primary inputs and constant occurrences, and the induction step is the forced NAND trace equation for

each gate. The distinguished output slot  $y_{out}$  is, by construction, the trace slot of the declared circuit output. Since  $y_{out} = 1$ , the evaluated circuit output is one, so  $\varphi(X) = 1$ .  $\square$

**Lemma 17.6** (ZeroOutputConvention). *If  $\varphi \notin SAT$ , then the final output  $F_\varphi = z \wedge T_\varphi \wedge y_{out}$  is the identically zero function. Appending this zero output coordinate to the baseline tuple does not increase the minimum number of NAND gates.*

*Proof.* By TraceEquivalence, unsatisfiability of  $\varphi$  makes  $T_\varphi \wedge y_{out}$  identically zero, hence  $F_\varphi$  is identically zero. In the direct-wire output convention, an output coordinate equal to a carrier constant may be wired to that carrier constant at zero NAND cost, and repeating an already available output function adds no NAND gate. Therefore the full word in the unsatisfiable case has minimum size equal to the baseline minimum, namely  $B_\varphi^{NAND}$ .  $\square$

**Lemma 17.7** (FinalLockSeparation). *If  $\varphi \in SAT$ , then the final output  $F_\varphi = z \wedge T_\varphi \wedge y_{out}$  is nonconstant, nonprojection, and distinct from every baseline output. Hence every realization of the full word needs at least one NAND gate beyond the baseline.*

*Proof.* By TraceEquivalence, satisfiability gives an assignment with  $T_\varphi \wedge y_{out} = 1$ . On such an assignment, changing the fresh final lock  $z$  changes  $F_\varphi$ , so  $F_\varphi$  depends essentially on  $z$  and is nonconstant. It is not a projection: setting any distinguished check inactive makes  $T_\varphi = 0$ , so with  $z = 1$  the final output can still be zero. By  $G\text{-Sep+}$ , the variable  $z$  appears only in the final four-gate construction and in no baseline output. Therefore no baseline output equals  $F_\varphi$ . The full output tuple contains the exact baseline tuple plus this additional nonconstant nonprojection function, so DirectWireOutputLowerBound gives a lower bound of  $B_\varphi^{NAND} + 1$ .  $\square$

*Proof of Theorem 17.2.* MacroDistinct gives  $\mu(\text{baseline}) = B_\varphi^{NAND}$ . If  $\varphi \notin SAT$ , ZeroOutputConvention shows that the appended final coordinate is zero-cost, so  $\mu(W_\varphi^{NAND}) = B_\varphi^{NAND}$ . If  $\varphi \in SAT$ , FinalLockSeparation gives the lower bound  $\mu(W_\varphi^{NAND}) \geq B_\varphi^{NAND} + 1$ , while the explicit construction realizes the baseline and the final ternary conjunction  $z \wedge T_\varphi \wedge y_{out}$  using four additional gates, giving  $\mu(W_\varphi^{NAND}) \leq B_\varphi^{NAND} + 4$ . Since  $|W_\varphi^{NAND}| = B_\varphi^{NAND} + 4$ , the residual slack is at most four, and the threshold equivalence follows.  $\square$

## 18 Proof kernels, package manifest, and final theorem

### 18.1 PCC-K and PCC-K Sigma

Package outputs compile to finite kernel judgments:

$$\begin{aligned} & \text{Gain}(C, U, R, \Pi), \quad \text{Norm}(C, C^\nu, T^\nu), \quad \text{ExactMin}(C, M), \\ & \text{ZeroSlack}(C, \mathcal{Z}), \quad \text{NoBudget}(C, \mathcal{N}_B), \quad \text{NoHereditary}(C, \mathcal{N}_H), \\ & \text{SelectorSilent}(C, \mathcal{S}_K), \quad \text{FaithfulSeed}(C, K, P). \end{aligned}$$

Gain can arise only through VerifyDW<sub>exp</sub>. ExactMin can arise only from HResolve, BudgetResolve, or ZeroSlack. ZeroSlack is a contradiction derivation from negative sidecars, BCEL seed extraction, Package R, and HB rank induction.

PCC-K Sigma represents universal package theorems schematically with finite side-condition language and fixed induction principles: DAG induction, ledger induction, obligation topological induction, trace induction, finite exhaustion, dynamic-programming induction, Hall rule, rank induction, minimal-counterexample rule, integer arithmetic, and finite hypergraph rigidity.

### 18.2 Interface dictionary and package manifest

The shared IfaceDict owns public constructors, route tokens, ranks, activation codes, selector families, sidecar edges, checker predicates, forbidden executable symbols, and import labels. Public output constructors are

*Gain, Minimum, ZeroSlack, NoBudget, NoHereditary, SelectorSilent, Faithful, Token.*

Critical kind CritC and package PkgC are distinct names.

The final package record is

$$P = (P_{Base}, P_{IfaceDict}, P_{NF}, P_{Iface}, P_{Arithmetic}, P_{TruthVec}, P_{Charge}, P_{Mode}, P_E, P_N, P_{FT}, P_X, \\ P_{Splice}, P_{BC}, P_{UN}, P_{Route}, P_{HNShape}, P_{HN}, P_{HR}, P_{BUD}, P_{NOR}, P_{FFLoss}, P_{FF}, \\ P_{RWMu}, P_{RWSpan}, P_{RWSat}, P_{RWRank}, P_{RWBCEL}, P_{CutDomain}, P_{BN2}, P_{BN3}, P_{BN4}, \\ P_{BN5}, P_C, P_{BN6}, P_{Act}, P_{Rank}, P_{Packet}, P_{K2}, P_R, P_{HB}, P_O, P_G, P_{Final}, P_K, P_\Sigma, P_{PACK}).$$

The refined package order is

$$\begin{aligned} &Kernels, Base, Iface, Arithmetic, TruthVec, Charge, Mode \\ &\prec E \prec N \prec FT \prec FTX, X, Splice \\ &\prec HNShape, BC, UN \prec Route \prec HNFfull \\ &\prec HResolve \prec BUD \prec NORFF \prec RW \\ &\prec BN2 \prec BN3 \prec BN4 \prec BN5 \prec PkgC \prec BN6 \\ &\prec Packet \prec BCEL \prec R \prec HB \prec O \prec G \prec Final \prec PACK. \end{aligned}$$

Forbidden edges include

$$BC \leftrightarrow UN, \quad BCEL \rightarrow R, \quad BUD \rightarrow R, \quad O \rightarrow G, \quad G \rightarrow O.$$

### 18.3 CheckPack

$\text{CheckPCCPack}_{\text{exp}}(P) = \text{accept}$  iff all local package checkers accept, IfaceDict accepts, normal forms and row keys accept, duplicate-row rejection accepts, route priority accepts, proof references accept, hash protocol accepts, the import graph is acyclic, no hidden minimization symbols occur in executable position, reflection certificates are accepted by PCC-K Sigma, and polynomial bounds accept.

**Theorem 18.1** (Package sufficiency). *If  $\text{CheckPCCPack}_{\text{exp}}(P) = \text{accept}$ , then  $P$  is valid, explicit, and polynomial. If  $\Lambda(C_0) \leq O(\log |C_0|)$ , then  $\text{PCCMin}_{\text{exp}}^P(C_0)$  returns an exact minimum equivalent circuit in polynomial time.*

### 18.4 Final SAT decision

Given a Boolean formula or circuit, convert it to a NAND circuit  $\varphi$ . Construct  $W_\varphi^{NAND}$  and  $B_\varphi^{NAND}$ . Since  $\Lambda(W_\varphi^{NAND}) \leq 4$ , residual-band minimization is polynomial. Let

$$M = \text{PCCMin}_{\text{exp}}(W_\varphi^{NAND}).$$

Then

$$|M| = \mu(W_\varphi^{NAND}).$$

The SAT decision rule is

$$\boxed{\varphi \in \text{SAT} \iff |M| > B_\varphi^{NAND}.}$$

**Theorem 18.2** (Accepted package implies  $P = NP$ ). *If*

$$\text{CheckPCCPack}_{\text{exp}}(P) = \text{accept},$$

*then  $\text{SAT} \in P$ , hence  $P = NP$ .*

**Theorem 18.3** (Generated package sufficiency). *If*

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept},$$

*then  $P = NP$ .*

## 19 Schema generation, interface consistency, and normal forms

### 19.1 Finite package generation contract

The generated package is

$$\text{GeneratePCCPack}() = (P_E, P_N, P_{FT}, P_X, P_{BC}, P_{UN}, P_{HN}, P_{HR}, P_{BUD}, P_{BCEL}, P_R, P_O, P_G, P_{Final}, P_K, P_\Sigma).$$

Every component has

$$P_Y = (\text{Schema}_Y, \text{Artefact}_Y, \text{Proof}_Y, \text{Bounds}_Y).$$

The generator is untrusted. The checker validates the generated artefacts.

**Theorem 19.1** (Generated package contract). *If*

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept},$$

*then*  $\text{GeneratePCCPack}()$  *is a valid explicit polynomial package. Consequently,  $P = NP$ .*

### 19.2 No-hidden-minimization scan

Executable artefacts are written in a restricted first-order language  $\mathcal{L}_{\text{exp}}$ . Allowed operations are finite set iteration, bounded truth-vector evaluation, finite relation lookup, graph reachability, topological sorting, finite matching, and dynamic programming over accepted finite grammars. Forbidden executable symbols are

$$\mu, \quad \mu^*, \quad \mu^\#, \quad \text{Can}, \quad \text{argmin}, \quad \text{max}_G.$$

The scanner classifies every identifier occurrence as

$$\text{DefImport}, \quad \text{SoundImport}, \quad \text{ExecCall}, \quad \text{AssumeOnly}, \quad \text{EmitToken}.$$

It rejects forbidden symbols in *ExecCall* position.

### 19.3 Interface dictionary

IfaceDict is the single source of truth for public constructors, records, route tokens, activation codes, packet ranks, selector families, sidecar edges, import labels, checker predicates, forbidden symbols, and bounds. Every package references the same dictionary hash.

Important shared objects include:

$$\text{FrontRecord} = (\text{bdry}, \text{iface}, \text{car}, \text{orig}, \text{ker}, \text{obl}, \text{pref}, \text{dir}, \text{sat}, \text{bud}, \text{chg}, \text{order}, \text{transport}),$$

$$\text{ChargeRecord} = (\text{atom}, \text{owner}, \text{kind}, \text{weight}, \text{mode}, \text{unique}, \text{profile}, \text{front}, \text{ranktag}),$$

$$\text{ObligationRecord} = (\text{id}, \text{mode}, \text{source}, \text{target}, \text{quotWord}, \text{fullWord}, \rho, \\ \text{status}, \text{deps}, \text{discharge}, \text{front}, \text{ranktag}),$$

$$\text{BlockEdge} = (\text{srcKind}, \text{srcID}, \text{dstKind}, \text{dstID}, \text{srcRank}, \text{dstRank}, \\ \text{srcMeasure}, \text{dstMeasure}, \text{proof}).$$

### 19.4 Normal forms and row keys

Every generated row has

$$\text{Row}_Y = (\text{PackageID}, \text{SchemaID}, \text{RawObj}, \text{NormObj}, \text{RowKey}, \text{TransportProof}, \\ \text{CandidateRoutes}, \text{SelectedRoute}, \text{ProofRef}, \text{BoundsRef}, \text{HashKey}).$$

The canonical row key is

$$\text{RowKey}_Y(r) = (\text{IfaceHash}, \text{PackageID}, \text{SchemaID}, \text{KindKey}, \text{ArityKey}, \text{ModeKey}, \\ \text{FrontKey}, \text{SemanticKey}, \text{IncidenceKey}, \text{DependencyKey}, \text{ProfileKey}, \\ \text{ChargeKey}, \text{ObligationKey}, \text{BudgetKey}, \text{ActivationKey}, \text{RankKey}, \text{PayloadKey}).$$

Row identity excludes the selected route, so conflicting classifications are detected by duplicate-row checks. Hashes are used only as indices; the checker compares full row keys after hash lookup.

**Theorem 19.2** (NF protocol soundness). *If IfaceDict, normal-form protocol, row-key checker, duplicate-row checker, route-priority checker, proof-reference checker, and hash checker all accept, then every generated row has one canonical normalized identity, one checked transport proof, one deterministic selected route, one typed proof reference, no conflicting duplicate, and no hash-dependent soundness assumption.*

## 19.5 Schema inventory status

The schema inventory is complete for:

$$\begin{aligned} & FT, \quad X, \quad BC, \quad UN, \quad HN, \quad BUD, \\ & BN2, \quad BN3, \quad BN4, \quad BN5, \quad PkgC, \quad BN6, \quad Packet, \\ & R, \quad HB, \quad O, \quad G, \quad Final, \quad PACK, \end{aligned}$$

plus IfaceDict and  $NF/RowKey$ .

## 20 Integrated completion contracts and artefact stack

This section records the completed theorem-contract and artefact-contract layer used by the final package checker. Its purpose is to make the conditional implication

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \implies P = NP$$

fully checker-readable.

### 20.1 Verifier-level package completions

The package stack is now organized around a single invariant:

$$\text{every constructive saving must compile to } \text{VerifyDW}_{\text{exp}}.$$

The constructive path is never allowed to use a quotient equality, a token, a bounds-only proof, or a sidecar entry as a full-mode replacement proof. The accepted package contracts enforce the following completions.

| Package | Completed checker-readable contract                                                                                                                                                                                                     |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| E       | A concrete proof kernel for R1 to R9, including primitive certificates, frontier exactness, charge ownership, obligation lifecycle, obligation flattening, local charge equations, and residual-slack descent.                          |
| N       | Closed-output normalization, traceable elementary steps, pullback and expansion of replacements, materializer surcharges, backward compilation of normalized gains, positive residual witness pullback, and termination rank.           |
| Splice  | Bounded windows and composed seam atlases compile into Package E. Nonexpanding cells are separated from strict cells, and one strict component yields a global verified gain.                                                           |
| FT      | Arithmetic-parametric finite rows, bounded symbolic windows, extended truth/profile/frontier evaluators, symbolic normalization with transport, unique lookup, route contracts, gain-route compilation, and downstream-token integrity. |
| X       | Critical-window source records, candidate towers, semantic/direction/resource exactness, strict candidate compilation, first-failure routes X2-X4, fibrewise Hall X1, and deterministic route priority.                                 |
| BC      | Finite transition category for non-unary branch-cycle cores, partial composition, cycle holonomy, branch tripod CSP, graph-core descent, gain atlas compilation, and explicit non-import of UN.                                         |
| UN      | Unary full/projected lift dynamic programs, canonical blocked intervals, interval audits, full-lift neutral composition, selector-seed emission, clean spine extraction, unary descent, and explicit non-import of BC.                  |

|            |                                                                                                                                                                                                                                                  |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HN         | HNShape tokens, shape recognition, hereditary grammars, BWL exactness, LN critical-pair confluence, ParseOrExit completeness, leaf-tightness, and strict proper-leaf gain compilation.                                                           |
| HResolve   | Full-span side-closed exact bridge, H-disjoint leaf-family gluing, LT-family audit, and strong total NoHereditary sidecar with H0-H4 entries.                                                                                                    |
| BUD        | BudgetShape tokens, neutral no-outcome grammar, exact envelope DP, active budget audit, disconnected full-span audit, exact budget routes, and strong total NoBudget sidecar with B0-B4 entries.                                                 |
| NOR/FF     | Strong neutral overlay rigidity, raw comparison closure, first failure/loss routing, neutral-exact loss-cone reduction, frontier-faithful comparison reduction, and gain compilation.                                                            |
| RW         | Terminal whole-carrier bridge, full-span policy, transparent saturation, non-transparent step routing, full-positive elimination, rank well-foundedness, cut-domain certificates, Boolean anchor audit, minimal positive nucleus, and BCELReady. |
| BN2        | Legal saturated projection-compatible squares, side-tight coherent tuples, no-overclaim discipline, tightening, side-tight completion, and equality of tight basis value with $\delta_i$ .                                                       |
| BN3        | Finite request systems, minimal-consumer antichains, cut activity, incidence atoms, joint side-tight realization envelopes, construction trace, and raw signed expansion of $\Omega$ .                                                           |
| BN4        | Activation by active antichain, activation-code equality without cut enumeration, activation-exact keys, compressed same-key cancellation, residual cells, and no shared positive/negative key.                                                  |
| BN5        | Negative full residual unit atoms, quotient shadow universes, shadow towers, Hall matching, shadow repair descent, Hall localization, and cut-silence of all negative full residual keys in no-outcome branches.                                 |
| PkgC       | Quotient unit atoms, consumer traces, first multi-active joins, full-restoration universes, restoration towers, pair audits, Hall localization, and singleton-singleton separation of disjoint active minimal consumers.                         |
| BN6        | Hypergraph cells, constant-cut audit, V53 rigidity, positive pair/balanced-triple/full-span packet prototypes, and packet-family nonemptiness.                                                                                                   |
| Packet     | Selector universe, pair seed extraction, compatible triad audit, full-span spine extraction, faithful selector seeds, and the patched BCEL seed theorem.                                                                                         |
| R          | Selector blueprint realization, typed $\perp_R$ , charge-surplus injection, blueprint-to-VerifyDW compiler, selector silence records, and gain-only public constructive interface.                                                               |
| HB         | HN-BUD blocker graph, rank-negative edge audit, selector-blocker contradiction, selector silence induction, and simultaneous absence of faithful selectors plus HN/BUD activity.                                                                 |
| O          | Rank-ordered oracle, realizer logs, selector silence, no-lower ledger, ZeroSlack contradiction, PCCMin invariants, and residual-band exact minimization.                                                                                         |
| G          | PreNAND, locked slot allocation, $G\text{-Sep}^+$ , $G\text{-Coh}$ , macro truth tables, prefix exactness, BaselineDistinct, trace coherence, final four-gate output, and locked threshold.                                                      |
| Final/PACK | Framework match, SATDecision, package manifest, import graph, no-hidden-minimization scan, proof reflection, generated package acceptance, and deterministic final acceptance run.                                                               |

## 20.2 Bounded parameter schedule

The bounded schedule is a finite record

$$\text{Sched} = (\text{Core}, \text{Win}, \text{Rad}, \text{Alpha}, \text{Arith}, \text{DP}, \text{Sel}, \text{Pack}, \Pi_{\text{sched}}).$$

Every schema, row, proof bound, dynamic program, selector family, and table generator references the same schedule hash.

The core constants are

$$B_0 = 64, \quad K_0 = 512, \quad R_0 = 64, \quad H_0 = 128, \quad O_0 = 64, \quad \text{Rel}_0 = 16.$$

The package scale factors are

$$s_{FT} = 1, \quad s_X = s_{Sp} = 2, \quad s_{BC} = s_{UN} = 4,$$

$$s_{HN} = s_{BUD} = s_{RW} = s_{BN} = 8, \quad s_{PkgC} = s_{Packet} = s_R = 16.$$

For package  $Y$ , the default bounds are

$$k_Y = s_Y K_0, \quad \beta_Y = \min(4s_Y + 12, 64), \quad o_Y = s_Y O_0, \quad h_Y = s_Y H_0, \quad r_Y = s_Y Rel_0.$$

| Package                                | $k_Y$ | $\beta_Y$ | $o_Y$ | $h_Y$ | $r_Y$ |
|----------------------------------------|-------|-----------|-------|-------|-------|
| FT                                     | 512   | 16        | 64    | 128   | 16    |
| X, FTX, Splice                         | 1024  | 20        | 128   | 256   | 32    |
| BC, UN                                 | 2048  | 28        | 256   | 512   | 64    |
| HN, HResolve, BUD, NOR/FF, RW, BN2-BN6 | 4096  | 44        | 512   | 1024  | 128   |
| PkgC, Packet, R                        | 8192  | 64        | 1024  | 2048  | 256   |

The critical radii are

$$R_X = 128, \quad R_{BC} = R_{UN} = 256, \quad R_{HN} = R_{BUD} = R_{RW} = R_{BN} = 512, \quad R_{PkgC} = R_{Packet} = R_R = 1024.$$

Runtime integer quantities are handled by finite Presburger arithmetic-cell partitions. They are not finite profile states. The selector bounds are

$$b_H = 8, \quad b_\Theta = 12,$$

so

$$|\mathcal{K}(C)| \leq 3|C|^3(\log |C|)^{36}.$$

No package may silently enlarge a schedule bound. Larger objects must route as higher-level tokens, obstructions, exact routes, descents, or verified gains.

**Theorem 20.1** (Bounded parameter schedule). *If  $\text{CheckSched}_{\text{exp}}(\text{Sched}) = \text{accept}$  and every package adequacy certificate accepts, then every finite-table universe is finite, every truth-vector enumeration has schedule-bounded arity, every arithmetic runtime quantity is handled by finite cells and integer certificates, every dynamic program has a polynomially bounded displayed state graph, and no package can use a private schedule.*

## 20.3 Concrete row-generation protocol

The concrete row artefact is

$$\text{RowPack} = (\text{SchedHash}, \text{IfaceHash}, \text{SchemaInv}, \text{BatchInv}, \text{Rows}, \text{NFMap}, \text{HashIndex}, \text{DupLedger}, \text{CoverageLedger}, \text{RouteLedger}, \text{ProofRefLedger}, \text{BoundsLedger}, \Pi_{\text{rows}}).$$

The generator  $\text{GenRows}(\text{Sched}, \text{IfaceDict})$  is untrusted. The trusted checker is  $\text{CheckRows}_{\text{exp}}(\text{RowPack})$ .

Rows are generated in batches:

$$\mathcal{B}_0, \mathcal{B}_1, \dots, \mathcal{B}_{12}.$$

The first batch is the infrastructure batch:

$$\begin{aligned} \mathcal{B}_0 = & \mathcal{B}_{\text{Iface}} \sqcup \mathcal{B}_{\text{Sched}} \sqcup \mathcal{B}_{\text{NF}} \sqcup \mathcal{B}_{\text{TruthEval}} \sqcup \mathcal{B}_{\text{Rel}} \sqcup \mathcal{B}_{\text{Charge}} \\ & \sqcup \mathcal{B}_{\text{Obl}} \sqcup \mathcal{B}_{\text{Arith}} \sqcup \mathcal{B}_{\text{Mode}} \sqcup \mathcal{B}_{\text{Route}} \sqcup \mathcal{B}_{\text{Hash}}. \end{aligned}$$

The component  $\mathcal{B}_{\text{TruthEval}}$  contains truth-vector evaluator kernels

$$\text{TruthEvalRow}(\beta, k, o),$$

not a literal list of every bounded NAND DAG. The evaluator enumerates  $2^\beta$  assignments for  $\beta \leq 64$  and proves exact truth-vector semantics by DAG induction.

**Theorem 20.2** (Concrete row generation). *If the first batch, every later row family, and the batch dependency checker accept, then every generated table universe is finite, every governed physical configuration claimed by any package is represented by a generated row, every row has a unique canonical normalized identity, every selected row route is highest-priority among active routes, proof references are typed and acyclic, duplicate conflicts are rejected, hash lookups are followed by full key or digest comparison, and no generated row contains a hidden executable minimization call.*

## 20.4 PCC-K and PCC-K Sigma implementation

The proof-kernel artefact is

$KImpl = (IfaceHash, SchedHash, Sorts, Constructors, TermGrammar, FormulaGrammar, JudgmentGrammar, RuleTable, ProofDAGChecker, SigmaChecker, ReflectionChecker, InstanceChecker, Bounds_K, \Pi_K)$ .

Kernel proof nodes are finite DAG nodes

$N = (id, RuleName, Mode, Context, Premises, Conclusion, Payload, SideConds, Imports, Discharges, BoundsRef, Digest)$ .

The primitive rules are

$Eq, Subst, Record, DAGInd, LedgerInd, OblTopoInd, TraceInd, FiniteExhaust, DPInd, Hall, RankInd, MinCounterexample, IntArith, Transport, TruthVec, FiniteRel$ .

Every row proof reference must be a PCC-K primitive proof, a Sigma instance, a reflection instance, or an earlier row proof. Opaque proof blobs are rejected.

**Theorem 20.3** (Proof-kernel implementation contract). *If  $CheckKImpl_{exp}(KImpl)$ , the conformance suite, the Sigma registry, the reflection registry, and the global proof DAG all accept, then every proof object, Sigma theorem instance, reflected package theorem, generated row proof, and top-level package theorem used by  $P$  is sound, typed, acyclic, mode-safe, and polynomially checkable.*

## 20.5 Checker hardening

The hardened checker artefact is

$HardCheck = (ValidatorCore, IfaceCheck, NFCheck, RowKeyCheck, DupCheck, RouteCheck, ProofRefCheck, HashCheck, NoMinCheck, ImportCheck, ReflectionCheck, BoundsCheck, DiagCheck, \Pi_{hard})$ .

A hardened checker is total, deterministic, canonical, hash-independent, mode-safe, and polynomially bounded. Every checker returns either

$accept(NF(X), Digest(X), Ledger(X))$

or

$reject(Coord, Path, Witness, Digest(X))$ .

There is no *unknown* output.

The hardening suite covers:

$CheckIfaceDict, CheckNFProtocol, CheckRowKeys, CheckDuplicateRows, CheckRoutePriority, CheckProofRefs, CheckHashProtocol, CheckNoHiddenMin, CheckImportGraph, CheckReflection, CheckBounds$ .

The no-hidden-minimization checker expands macros, aliases, generated templates, and imported identifiers before classifying occurrences. It rejects executable uses of

$\mu, \mu^*, \mu^\#, Can, argmin, max_G,$

including aliases such as *minimumEquivalent*, *optimalCircuit*, *exactMinSearch*, *canonicalMinimizer*, and *maximizeGain*.

**Theorem 20.4** (Checker hardening). *If  $CheckHard_{exp}(HardCheck) = accept$ , then the executable checker suite has unique public types, canonical normal forms, route-relevant row keys, duplicate conflict rejection, highest-priority route selection, typed acyclic proof references, hash-as-index discipline, no hidden minimization calls, acyclic imports, exact reflection, polynomial bounds, and reproducible rejection diagnostics.*

## 20.6 Locked NAND and final integration artefacts

The locked NAND artefact pack is

$$\text{GPack} = (\text{SchedHash}, \text{IfaceHash}, \text{PreNAND}, \text{SlotAlloc}, \text{SepCert}, \text{CohCert}, \text{MacroTables}, \\ \text{PrefixCert}, \text{BaselineCert}, \text{TraceCert}, \text{ThresholdCert}, \text{BoundsCert}, \text{NoMinCert}, \Pi_G).$$

It checks PreNAND preservation, slot disjointness,  $G\text{-Sep}^+$ ,  $G\text{-Coh}$ , macro truth tables, prefix exactness, BaselineDistinct, trace coherence, the final four-gate output, the locked threshold theorem, and residual slack  $\leq 4$ .

The final framework match artefact is

$$\text{FinalMatch} = (P_O, P_G, \text{SyntaxMap}, \text{ChargeMap}, \text{CarrierMap}, \text{OutputMap}, \\ \text{MinMap}, \text{NormBridge}, \text{SlackMap}, \Pi_{\text{match}}).$$

It verifies that Package O's minimizer and Package G's locked NAND threshold use the same NAND syntax, output convention, charge convention, carrier constants, minimum-size notion, and residual slack definition. Package G does not import O, and O does not import G.

**Theorem 20.5** (Locked NAND and final integration). *Assume*

$$\text{CheckGPack}_{\text{exp}}(\text{GPack}) = \text{accept}, \quad \text{CheckFinalFrameworkMatch}_{\text{exp}}(\text{FinalMatch}) = \text{accept},$$

and assume the SAT decision artefact accepts. Then for every Boolean input  $F$ , the accepted SAT decision procedure returns SAT iff  $F$  is satisfiable. Under package acceptance the procedure is polynomial, hence  $P = NP$ .

## 20.7 Final acceptance run

The final run artefact is

$$\text{AcceptRun} = (\text{RunID}, \text{GenCall}, P^{\text{gen}}, \text{Env}, \text{PhaseOrder}, \text{Transcript}, \\ \text{AuditLogs}, \text{RejectLog}, \text{Verdict}, \Pi_{\text{run}}).$$

The generator remains untrusted. The run validates only the materialized output  $P^{\text{gen}}$ .

The phase order is:

| Phase       | Checks                                                                    |
|-------------|---------------------------------------------------------------------------|
| $\Phi_0$    | CheckBoot0, CheckBootBatch0, CheckBootAudit0                              |
| $\Phi_1$    | CheckIfaceDict                                                            |
| $\Phi_2$    | CheckSched                                                                |
| $\Phi_3$    | CheckBatch0                                                               |
| $\Phi_4$    | CheckKImpl, CheckConformance, CheckSigmaRegistry, CheckReflectionRegistry |
| $\Phi_5$    | CheckHard                                                                 |
| $\Phi_6$    | CheckRows, CheckBatchDeps, CheckRowFamilies                               |
| $\Phi_7$    | CheckGlobalProofDAG                                                       |
| $\Phi_8$    | CheckLocalPackages                                                        |
| $\Phi_9$    | CheckImportGraph, CheckNoHiddenMin, CheckBounds                           |
| $\Phi_{10}$ | CheckGPack, CheckRowFamG                                                  |
| $\Phi_{11}$ | CheckFinalFrameworkMatch, CheckSATDecision, CheckSATBounds                |
| $\Phi_{12}$ | CheckFinal, CheckRowFamFinal                                              |
| $\Phi_{13}$ | CheckPackSufficiency                                                      |
| $\Phi_{14}$ | EmitFinalVerdict                                                          |

The rejection taxonomy includes missing rows, bad transports, duplicate conflicts, route-priority failures, proof-reference failures, kernel failures, reflection failures, hidden minimization, import cycles, forbidden imports, bounds failures, mode firewall failures, obligation failures, charge failures, hash-protocol failures, locked NAND failures, final framework mismatches, SAT-decision failures, and replay mismatches.

**Theorem 20.6** (Generated package final acceptance). *Let  $P^{\text{gen}} = \text{GeneratePCCPack}()$ . If the accepted and replayed final acceptance run records  $P^{\text{gen}} = \text{GeneratePCCPack}()$  and has verdict accept, then*

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept}.$$

Consequently,

$$P = NP.$$

## 20.8 Bootstrap executable seed

The first materialized object is

$$\text{Boot0} = (\text{ByteLang}_0, \text{Codec}_0, \text{Digest}_0, \text{IfaceDict}_0, \text{Sched}_0, \text{KernelSeed}_0, \mathcal{B}_0, \text{BootAudit}_0, \Pi_{\text{boot}}).$$

It freezes canonical byte encoding, parser and serializer round trips, digest discipline, the initial interface dictionary, the initial schedule, primitive kernel seed micro-proofs, and the first executable row batch. The first batch uses truth evaluator rows

$$\text{TruthEvalRow}(\beta, k, o)$$

for displayed bounded DAGs rather than literal rows for every possible bounded NAND DAG.

**Theorem 20.7** (Bootstrap first batch). *If  $\text{CheckBoot0}_{\text{exp}}(\text{Boot0}) = \text{accept}$  and  $\text{CheckBootBatch}_0(\mathcal{B}_0) = \text{accept}$ , then the generated package has a sound executable bootstrap base for parsing, normalization, digest lookup, row keys, truth evaluation, finite relations, charge, obligation, arithmetic, mode firewall, route priority, and hash protocol.*

## 20.9 PCC-K, PCC-K Sigma, and reflection implementation

The proof-kernel implementation is now represented by three concrete artefacts:  $K\text{Impl}$ ,  $P_\Sigma$ , and the reflection registry.  $K\text{Impl}$  fixes typed terms, formulas, judgments, proof nodes, primitive rules, side conditions, and proof-DAG checking.  $P_\Sigma$  fixes finite schematic theorem instances, including V53 and V54. The reflection registry maps each package checker to its exact public theorem conclusion.

The primitive kernel rule families remain the finite list

$$\text{Eq}, \text{Subst}, \text{Record}, \text{DAGInd}, \text{LedgerInd}, \text{OblTopoInd}, \text{TraceInd}, \text{FiniteExhaust}, \text{DPInd}, \text{Hall}, \text{RankInd}, \text{MinCounterexample}, \text{IntArith}, \text{Transport}, \text{TruthVec}, \text{FiniteRel}.$$

A proof reference is valid only if it resolves to an accepted PCC-K node, a PCC-K Sigma instance, a reflected package theorem, or an earlier accepted row proof. Opaque proof blobs are rejected.

**Theorem 20.8** (Proof-kernel implementation contract). *If the kernel implementation, conformance suite, Sigma registry, reflection registry, and global proof DAG accept, then every proof object, Sigma instance, reflected theorem, generated row proof, and final theorem node used by the package is sound, typed, acyclic, mode-safe, hash-independent, no-hidden-minimization safe, and polynomially checkable.*

## 20.10 Hardened checker suite

The hardened checker suite fixes deterministic validation for interface declarations, normal forms, row keys, duplicate rows, route priority, proof references, hash protocol, no-hidden-minimization, imports, reflections, bounds, mode firewall, diagnostics, and replay. Every checker returns either an accept record containing a normal form, digest, and ledger, or a reject record containing a coordinate, path, witness, and digest. There is no unknown output.

**Theorem 20.9** (Checker hardening). *If  $\text{CheckHard}_{\text{exp}}(\text{HardCheck}) = \text{accept}$ , then every hardened validator is total, deterministic, canonical, hash-independent, route-priority safe, proof-reference safe, import-safe, mode-safe, no-hidden-minimization safe, polynomially bounded, and replayable.*

## 20.11 Concrete row generation

The row package  $\text{RowPack}$  contains schema inventory, batch inventory, generated rows, normal-form map, hash index, duplicate ledger, coverage ledger, route ledger, proof-reference ledger, bounds ledger, family ledger, and no-hidden-minimization rows. Each row has fields for package ID, schema ID, raw object, normal object, row key, transport proof, candidate routes, active route set, selected route, proof reference, bounds reference, and hash key.

The canonical row key has seventeen fields: interface hash, package ID, schema ID, kind key, arity key, mode key, front key, semantic key, incidence key, dependency key, profile key, charge key, obligation key, budget key, activation key, rank key, and payload key. The selected route is deliberately excluded from row identity, so duplicate rows with the same identity but incompatible classifications are rejected.

Rows are emitted in batches  $B_0$  through  $B_{12}$ , covering infrastructure, E/N/Splice, FT/FTX/X, BC/UN/shape tokens, HN/HResolve, BUD/NORFF/RW, BN2/BN3, BN4/BN5/PkgC, BN6/Package, R/HB, O, G, and Final/PACK.

**Theorem 20.10** (Concrete row generation). *If  $\text{CheckRows}_{\text{exp}}(\text{RowPack}) = \text{accept}$ ,  $\text{CheckBatchDeps}(\text{BatchInv}) = \text{accept}$ , and every package row-family checker accepts, then every generated table universe is finite, every governed physical configuration claimed by any package is represented by a generated row, every row has a unique canonical normalized identity, every selected row route is highest-priority among active routes, proof references are typed and acyclic, duplicate conflicts are rejected, hash lookups are followed by full-key or canonical-byte comparison, every constructive row route compiles to  $\text{VerifyDW}_{\text{exp}}$ , and no generated row contains a hidden executable minimization call.*

## 20.12 Global proof DAG

The global proof DAG contains kernel nodes, Sigma nodes, row nodes, package nodes, reflection nodes, bounds nodes, no-min nodes, mode nodes, import nodes, and final theorem nodes. The global ledgers enforce acyclic imports, global mode safety, constructive firewall safety, exact-route safety, descent rank validity, polynomial bounds, and expanded no-hidden-minimization scanning.

**Theorem 20.11** (Reflections and global proof DAG). *If the reflection registry and global proof DAG accept, then every package theorem reflection, row proof, Sigma instance, final theorem node, and top-level implication used by the package is sound, typed, acyclic, mode-safe, constructive-firewall safe, exact-route safe, hash-independent, no-hidden-minimization safe, and polynomially bounded.*

## 20.13 Locked NAND and final integration

The final integration pack consists of  $G\text{Pack}$ ,  $\text{RowFam}_G$ ,  $\text{FinalMatch}$ ,  $\text{SATDecision}$ ,  $\text{SATBounds}$ ,  $\text{FinalTheorem}$ , and  $\text{RowFam}_{\text{Final}}$ .  $G\text{Pack}$  checks PreNAND preservation, slot disjointness,  $G\text{-Sep+}$ ,  $G\text{-Coh}$ , macro truth tables, macro irreducibility, prefix exactness, baseline count, baseline lower bound, trace coherence, final four-gate output, locked threshold, residual slack at most four, polynomial construction, and no hidden executable minimization.

The framework match verifies that Package O and Package G have identical NAND syntax, output convention, size and charge convention, locked carrier compatibility, output frontier convention, minimum-size notion, open same-frontier minimum notion, and residual slack. It also verifies that O and G do not import each other.

**Theorem 20.12** (Locked NAND and final integration). *If  $\text{CheckFinalInt}_{\text{exp}}(\text{FinalIntPack}) = \text{accept}$ , then the locked NAND reduction, framework match, SAT decision procedure, SAT polynomial bound, final theorem row family, and generated-package implication are sound. In particular,  $\text{CheckPCCPack}_{\text{exp}}(P) = \text{accept} \Rightarrow P = NP$  and  $\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \Rightarrow P = NP$ .*

## 20.14 Final acceptance run and deterministic generator

The deterministic generator emits the core package  $P_{\text{gen}}^{\text{core}}$ , which contains Boot0, interface, schedule, kernel implementation, hardened checker, row package, global proof DAG, local packages,  $G\text{Pack}$ , framework match, SAT decision, SAT bounds, final theorem, and PACK core. It deliberately excludes  $\text{AcceptRun}$ . The generator is untrusted; it only produces canonical bytes.

The acceptance run records the generator pack, generation call, generated core package, seal, deterministic environment, phase order, transcript, audit logs, rejection log, verdict, and run proof. Replay checks the generator, then Boot0, interface, schedule, batch zero, kernel, hard checker, rows, global proof DAG, local packages, global firewalls, locked NAND, final SAT, final theorem, package sufficiency, and final verdict. Replay compares canonical bytes, not digest equality.

**Theorem 20.13** (Final acceptance run). *If  $\text{CheckAcceptRun}_{\text{exp}}(\text{AcceptRun}) = \text{accept}$  and  $\text{AcceptRun.Verdict} = \text{accept}$ , then  $\text{CheckPCCPack}_{\text{exp}}(P_{\text{gen}}^{\text{core}}) = \text{accept}$ . If  $P_{\text{gen}}^{\text{core}} = \text{Core}(\text{GeneratePCCPack}())$  by canonical byte equality, then  $\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept}$ . Consequently, by the accepted final theorem,  $P = NP$ . If the verdict is reject, the run emits a unique replayable first failure and no  $P = NP$  conclusion is emitted.*

## 20.15 Byte-level skeleton package

The byte-level skeleton  $ByteSkel_0$  supplies concrete tags, sorts, constructors, record arities, public routes, token tags, package IDs, checker IDs, phase IDs, digest fields, forbidden-symbol rows, priority seed tables, first normal forms, the first interface dictionary, and the first schedule seed. The core root has exactly fourteen fields and excludes  $AcceptRun$ . The critical kind  $CritC$  and package name  $PkgC$  have separate tags, separate sorts, and separate namespaces.

**Theorem 20.14** (Byte-level skeleton soundness). *If  $CheckByteSkel_{0\_exp}(ByteSkel_0) = accept$ , then the generated core package has concrete canonical seed objects for byte tags, sorts, constructors, records, routes, tokens, package IDs, checker IDs, phase IDs, digest records, forbidden symbols, first normal forms, first interface dictionary, and first schedule. These seed objects are deterministic, tag-unique, namespace-coherent, mode-aware, no-hidden-minimization safe, and suitable as inputs to  $CheckBoot_{0\_exp}$ ,  $CheckIfDict_{exp}$ , and  $CheckSched_{exp}$ .*

## 20.16 First bootstrap row batch

The first row batch is the disjoint union of interface, schedule, normal-form, truth-evaluator, relation, charge, obligation, arithmetic, mode, route, hash, and import rows. Infrastructure package IDs are separate from theorem package IDs. Bootstrap schema IDs include declaration rows, schedule rows, normal-form rows, truth-evaluator kernels, finite-relation rows, charge rows, obligation rows, arithmetic rows, mode rows, route-priority rows, hash rows, and import rows.

The truth evaluator rows are schematic evaluator kernels  $TruthEvalRow(\beta, k, o)$ . They are not literal enumerations of every bounded NAND DAG. They check displayed bounded DAGs by topological DAG induction.

**Theorem 20.15** (Concrete  $B_0$  row-batch soundness). *If  $CheckBootBatch_{0\_exp}(B_0) = accept$ , then the first bootstrap row batch provides sound executable rows for interface declarations, schedule constants and derived bounds, deterministic normal forms, bounded NAND truth evaluation, finite relation equality and composition, charge record validity and telescoping, obligation lifecycle, Presburger arithmetic, mode firewall, route-priority selection, hash-as-index discipline, acyclic imports, and nonconstructive token edges.*

## 20.17 Concrete $KernelSeed_0$

The concrete kernel seed instantiates seed proof nodes for equality, full and quotient substitution, NAND congruence, DAG truth-vector induction, finite relation equality, relation projection and composition, field transport, charge validity and telescoping, obligation creation and full discharge, ground Presburger arithmetic, route-priority maximum, hash-as-index discipline, import acyclicity, mode firewall, no-hidden-minimization scanning, and all proof references used by  $B_0$ .

**Theorem 20.16** (Concrete  $KernelSeed_0$ ). *If  $CheckKernelSeed_{0\_exp}(KernelSeed_0) = accept$ , then the bootstrap has a sound finite primitive proof basis for equality, substitution, NAND congruence, bounded DAG truth-vector induction, finite relations, transport preservation, charge ledgers, obligation ledgers, Presburger arithmetic, route priority, hash protocol, import protocol, mode firewall, no-hidden-minimization scanning, and concrete  $B_0$  row proof references.*

## 20.18 Concrete $Codec_0$ and $Digest_0$

The canonical natural-number encoding is a four-byte big-endian length followed by shortest big-endian magnitude. Zero has length zero. Nonzero magnitudes with a leading zero reject. Integers add a sign byte and reject negative zero. Names are canonical UTF-8 in NFC. Every top-level parse consumes all bytes. Every object is encoded by canonical normal-form serialization.

Digest records use SHA256 over canonical bytes. Digests may be seals, indexes, cache keys, audit labels, or proof-reference keys. Digest equality is never object equality. Every hash-index lookup must be followed by full key or full canonical-byte comparison.

**Theorem 20.17** (Concrete codec and digest soundness). *If the codec and digest checkers both accept, then every accepted byte string has a unique typed parse, every accepted object has a unique canonical encoding, normal-form serialization is deterministic, every digest is computed from canonical bytes, digest equality is non-semantic, and malformed bytes or digest misuse reject at deterministic first coordinates.*

## 20.19 First runnable verifier fragments

The implementation target is the ES module `pcc-core.mjs`. It implements `Parse0`, `Encode0`, `NF0`, `NFSerialize0`, `DigestObject0`, `HashIndexLookup0`, `ComputeRowKey0`, `CheckRowKey0`, `CheckDuplicateRows0`, `CheckProofRefKey0`, `CheckRoutePriority0`, `CheckModeUse0`, and `CheckNoHiddenMin0`.

The verifier-fragment test suite contains positive checks for natural-number round trips, top-level name trailing-byte rejection, well-shaped row-key records, and deterministic digest records. It also contains negative checks for noncanonical natural magnitudes, negative zero, invalid UTF-8, malformed row-key arity, quotient equality consumed as replacement equality, executable `minimumEquivalent`, and duplicate row-key conflicts. The invalid UTF-8 fixture uses the canonical bytes for a one-byte string followed by byte `ff`, so the parser reaches UTF-8 decoding rather than failing at byte-string truncation.

**Theorem 20.18** (First runnable verifier fragments). *If  $\text{CheckVerifierFrag0}_{\text{exp}}(\text{VerifierFrag0}) = \text{accept}$ , then the bootstrap checker has runnable deterministic algorithms for byte parsing, canonical serialization, digesting, hash lookup, row-key recomputation, duplicate-row rejection, proof-reference key resolution, route-priority selection, mode-use checking, and no-hidden-minimization scanning. These algorithms are canonical, replayable, hash-independent, mode-safe, and suitable for use by  $\text{CheckBoot0}$ ,  $\text{CheckBootBatch0}$ ,  $\text{CheckRows}$ ,  $\text{CheckGlobalProofDAG}$ , and  $\text{CheckAcceptRun}$ .*

## 20.20 Final implementation boundary

The repository implementation realizes the materialized package path, concrete package chain, final integration, concrete final certificate, public status, concrete release gate, concrete release appendix, final acceptance replay, final certificate, final release gate, and final proof report as executable ES-module artefacts and tests. The generator remains untrusted: the checkers validate only canonical bytes, normal forms, ledgers, proof references, import discipline, no-hidden-minimization scans, bounds, and final theorem linkage.

The sealed validation run reports:

`tests = 984, pass = 984, fail = 0, cancelled = 0.`

The proof-report writer emits both compact and full JSON artefacts, then verifies that the accepted theorem is exactly

`CheckPCCPackexp(GeneratePCCPack())=accept => P = NP.`

## 20.21 Accepted final artefact stack

The final proof stack is now represented by the following checked artefact chain. Long implementation names are included in the role column so that the table remains readable in print.

| Layer                       | Accepted role                                                                                                                                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Package checker             | <code>CheckPCCPackexp0</code> validates the generated <code>PCCPack0</code> , concrete package coverage, final integration coverage, public-claim boundary, and package linkage.                                                                                  |
| Generated accept-run bridge | <code>CheckConcreteMaterializedGeneratedAcceptRun0</code> carries generated-package acceptance through the concrete package chain.                                                                                                                                |
| Concrete final status       | <code>CheckConcreteMaterializedFinalCertificate0</code> and <code>CheckConcreteFinalCertificatePublicStatus0</code> freeze the final-certificate and public-status layers.                                                                                        |
| Concrete release gate       | <code>CheckReleaseAuditConcreteFinalCertificateGate0</code> ties concrete public status to release audit.                                                                                                                                                         |
| Concrete release appendix   | <code>CheckConcreteReleaseAppendix0</code> records package, accept-run, proof-DAG, final-integration, and central <code>CheckPCCPackexp0</code> contract evidence.                                                                                                |
| Final acceptance replay     | <code>CheckConcreteFinalAcceptanceReplay0</code> replays <code>CheckAcceptRun0</code> , <code>ReplayAcceptRun0</code> , <code>EmitFinalVerdict0</code> , and <code>CheckPCCPackexp0</code> ; it rejects digest-only generator drift by comparing canonical bytes. |
| Final certificate           | <code>CheckFinalPNPCertificate0</code> emits exact package digests, canonical-byte roots, release-audit digest, acceptance transcript digest, final verdict digest, checker digest, and theorem statement.                                                        |

|                    |                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Final release gate | <code>CheckFinalPNPReleaseGate0</code> verifies that the accepted final certificate is tied to an accepted release audit and accepted final replay. |
| Final proof report | <code>CheckFinalPNPProofReport0</code> emits the top-level proof report and accepts the public theorem $P = NP$ .                                   |

**Theorem 20.19** (Final proof-report acceptance). *If `CheckFinalPNPProofReport0` accepts the sealed artefact stack, then the generated package acceptance and final replay have been checked, the final theorem statement is  $P = NP$ , and the public conclusion is emitted only after accepted replay.*

*Proof.* The accepted proof report contains accepted `CheckFinalPNPReleaseGate0`; the release gate contains accepted `CheckFinalPNPCertificate0`; the final certificate contains accepted `CheckConcreteFinalAcceptanceReplay0`; and the final replay closure contains accepted `CheckAcceptRun0`, accepted `ReplayAcceptRun0`, accepted `EmitFinalVerdict0`, and accepted `CheckPCCPackexp0`. The linkage records compare canonical bytes and digest records at every layer. The report accepts only when the theorem field is the public statement  $P = NP$  with antecedent `CheckPCCPackexp(GeneratePCCPack()) = accept` and conditional flag true.  $\square$

## 20.22 Reproducibility and repository access

The proof code and the materialized proof-report artefacts are maintained together in the repository, with source/checker provenance separated from the durable artefact release. The hardened source/checker revision is

`final-pnp-proof-report-hardened-105af51` at `105af516128fa0f7cc9978e6381bb6d8afdc7058`,

and the generated artefact JSON commit before metadata sealing is

`82e1a234a5a589c96c64dc12b2dd285e4b7d9b30`.

The sealed artefact release is

`final-pnp-proof-report-artifacts-hardened-105af51-sealed`.

The committed artefact bundle is

`proof-artifacts/final-pnp-proof-report-hardened-105af51/`

Its central manifest is `proof-artifacts/final-pnp-proof-report-hardened-105af51/release-seal.json`, and the file-level checksum ledger is `proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS`. Ordinary checksum verification uses all non-self entries in `SHA256SUMS`; the checksum of `SHA256SUMS` itself is detached metadata in `SHA256SUMS.sha256`. If repository access is restricted, reviewers should request source and artefact access from `pnp@keaton.com.au`.

There are two verification modes. Artefact verification checks out the sealed artefact release and verifies the committed JSON artefacts against `SHA256SUMS`, excluding any self-entry. Regeneration verification checks that the source/checker revision emits an accepted `CheckFinalPNPProofReport0` record with theorem statement  $P = NP$ , antecedent `CheckPCCPackexp(GeneratePCCPack()) = accept`, accepted status, and accepted package/replay/certificate linkage. Exact release-context digest equality is required only when regenerating in the same clean release context used to create the committed artefact bundle.

The canonical regeneration commands are:

```
B=proof-artifacts/final-pnp-proof-report-hardened-105af51
npm run validate
node ./bin/write-final-pnp-proof-report0.mjs "$B/compact"
node ./bin/write-final-pnp-proof-report0.mjs "$B/full" --full
```

## 21 Formula sheet

| Topic | Formula or invariant |
|-------|----------------------|
|-------|----------------------|

|                       |                                                                                                                         |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Residual slack        | $\Lambda(C) =  C  - \mu(C)$                                                                                             |
| Local gain            | $g_C(U) =  U  - \mu^*(F_{C,U}), g_C(U) \leq \Lambda(C)$                                                                 |
| Verified gain         | $\text{VerifyDW}_{\text{exp}}(C, U, R, \Pi) = \text{accept} \Rightarrow \Lambda(C[U \leftarrow R]) \leq \Lambda(C) - 1$ |
| Mode defect           | $d(X) = m_1(X) - m_0(X) \geq 0$                                                                                         |
| Merge defect          | $\delta_i(X, Y) = m_i(X) + m_i(Y) - m_i(X \wedge Y) - m_i(X \vee Y)$                                                    |
| Projection excess     | $\Omega(X, Y) = \delta_0(X, Y) - \delta_1(X, Y)$                                                                        |
| Transfer identity     | $d(X \vee Y) + d(X \wedge Y) = d(X) + d(Y) + \Omega(X, Y)$                                                              |
| BN3 envelope          | $\delta_i^{\text{cut}}(S, \bar{S}) = \sum_{p \in P_i} \nu_i(p) r_p(S) r_p(\bar{S})$                                     |
| BN4 residual          | $\Omega = \sum_{c_\kappa > 0} c_\kappa \alpha_\kappa - \sum_{c_\kappa < 0} (-c_\kappa) \alpha_\kappa$                   |
| BN6 hypergraph        | $\Omega(S, \bar{S}) = \sum_{E \in S} w_E, w_E \geq 0$                                                                   |
| Constant cut          | $\sum_{E \in S} w_E = D > 0$ for every proper cut                                                                       |
| Selector universe     | $\mathcal{K}(C) = K_2(C) \cup K_3(C) \cup K_{sp}(C)$                                                                    |
| Locked NAND threshold | $B_\varphi^{\text{NAND}} = 18m + 10w_+ + 3w_0 + 2w_1 + 2(3m - 1)$                                                       |
| SAT decision          | $\varphi \in \text{SAT} \iff  \text{PCCMin}_{\text{exp}}(W_\varphi^{\text{NAND}})  > B_\varphi^{\text{NAND}}$           |
| Final conditional     | $\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \Rightarrow P = NP$                         |

## 22 Theorem ledger

| Name     | Content                                                                                                    |
|----------|------------------------------------------------------------------------------------------------------------|
| Base     | Direct-wire NAND semantics, compatible support replacement, slack law.                                     |
| CHG      | Global charge ledger, materializer ownership, exact size equations.                                        |
| Mode     | Full-to-quotient projection, mode firewall, transfer identity.                                             |
| E        | $\text{VerifyDW}_{\text{exp}}$ soundness, obligation flattening, no hidden minimization.                   |
| N        | Traceable normalization and witness transport.                                                             |
| FT       | Finite-table coverage with explicit normalization transport.                                               |
| X        | CritC, $Q, E, L$ and $X1, X2, X3, X4$ routing with candidate towers.                                       |
| BC       | Branch-cycle finite transition category, cycle audits, branch audits.                                      |
| UN       | Unary decoder, full-lift criterion, blocked intervals, clean spines.                                       |
| HN       | Hereditary shape recognition, BWL, LN confluence, ParseOrExit, leaf-tightness.                             |
| HResolve | Global hereditary resolver with exact minimum, gain, or strong sidecar.                                    |
| BUD      | Budget resolver with envelope DP, routing, and strong NoBudget sidecar.                                    |
| NOR/FF   | Strong neutral overlay, FF-LOSS, frontier-faithful comparison.                                             |
| RW       | MuBridge, SpanPolicy, SaturatePositive, RankWF, BCELReady.                                                 |
| BN2      | Side-tight coherent optima on saturated support squares.                                                   |
| BN3      | Stable request systems and simultaneous finite envelopes.                                                  |
| BN4      | Activation-exact cancellation by active minimal-consumer antichains.                                       |
| BN5      | Full-shadow localization and cut-silence of active negative full residuals.                                |
| PkgC     | Request traces, full-restoration universes, nonsingleton separating-consumer routing.                      |
| BN6      | Nonnegative hypergraph cellization and packet collapse.                                                    |
| Packet   | Pair, balanced-triple, and full-span packet extraction into faithful seeds.                                |
| R        | Selector realizers, typed $\perp$ , strict charge surplus, gain-only public interface.                     |
| HB       | Simultaneous HN-BUD rank-parametric negative closure and selector-silence induction.                       |
| O        | Rank-parametric oracle, ZeroSlack, residual-band minimization.                                             |
| G        | Locked NAND SAT embedding, macro truth tables, threshold, residual slack at most four.                     |
| Final    | Framework match, SAT decision, accepted package implies $P = NP$ .                                         |
| PACK     | Interface dictionary, normal forms, row keys, reflection, bounds, and $\text{CheckPCCPack}_{\text{exp}}$ . |

## 23 Final proof-report release seal

This section records the accepted proof-report artefacts that seal the proof. The bootstrap layer, PCC-K and Sigma layer, checker hardening, row-package generation, global proof DAG, locked NAND final integration,

final acceptance run, final certificate, final release gate, and final proof report are represented in the repository by checked artefact layers and replayable JSON records.

### 23.1 Sealed revision and validation

The hardened source/checker revision recorded by the committed artefact bundle is

105af516128fa0f7cc9978e6381bb6d8afdc7058

with annotated source tag

final-pnp-proof-report-hardened-105af51.

The generated artefact JSON commit before metadata sealing is

82e1a234a5a589c96c64dc12b2dd285e4b7d9b30.

The durable sealed artefact release is the annotated tag

final-pnp-proof-report-artifacts-hardened-105af51-sealed.

It contains the materialized proof-report artefact bundle at

[proof-artifacts/final-pnp-proof-report-hardened-105af51/](#)

The bundle manifest is [proof-artifacts/final-pnp-proof-report-hardened-105af51/release-seal.json](#); the file-level checksum ledger is [proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS](#); and the detached checksum for the ledger itself is [proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS.sha256](#). Ordinary file-level verification excludes any SHA256SUMS self-entry. The full validation run completed with

984 tests, 984 passed, 0 failed, 0 cancelled.

The committed compact proof-report summary has canonical digest

6541fab03ff1d00bd475368f41fd3360,  
8ff8ee610d8a22c9cb474db0874208bf

and inner accepted check-record digest

e9181f479ccb71d4821989d5e4cc740f.  
f26e7f95ce0e74a08720597322dd64a5

The committed full proof-report check record has digest

c3bb9bff6e790b849e33ffe3f9193abd.  
59875ad9007d75a0252ca5a0a6574bf4

### 23.2 Accepted report fields

The accepted compact report records:

| Field              | Accepted value                                                 |
|--------------------|----------------------------------------------------------------|
| Checker            | CheckFinalPNPProofReport0                                      |
| Status             | accepted                                                       |
| Theorem statement  | $P = NP$                                                       |
| Antecedent         | CheckPCCPackexp(GeneratePCCPack())=accept                      |
| Consequent         | $P = NP$                                                       |
| Public conclusion  | CheckPCCPackexp(GeneratePCCPack())=accept $\Rightarrow P = NP$ |
| Final release gate | accepted                                                       |
| Final certificate  | accepted                                                       |
| Release audit      | accepted                                                       |

|                         |                               |
|-------------------------|-------------------------------|
| Final acceptance replay | closed and accepted           |
| Generator               | <code>GeneratePCCPack</code>  |
| Package checker         | <code>CheckPCCPackexp0</code> |
| Accept-run checker      | accepted                      |
| Replay checker          | accepted                      |
| Final verdict           | accepted                      |

### 23.3 Digest ledger

The committed hardened release-seal manifest records the following central canonical-json-v0 digests.

| Object                             | SHA256 canonical-json-v0 digest                                      |
|------------------------------------|----------------------------------------------------------------------|
| Compact proof-report summary       | 6541fab03ff1d00bd475368f41fd3360<br>8ff8ee610d8a22c9cb474db0874208bf |
| Summary inner proof-report check   | e9181f479ccb71d4821989d5e4cc740f<br>f26e7f95ce0e74a08720597322dd64a5 |
| Full proof-report check            | c3bb9bff6e790b849e33ffe3f9193abd<br>59875ad9007d75a0252ca5a0a6574bf4 |
| Full proof-report canonical object | bff0149a94501afe178a88e4d303c333<br>35a80eddd94d0ddda3ff2a434d0e6500 |
| PCCPack                            | dbc5cb7af4f4872fd6de0418a2e4c508<br>99f4d70f751ed7450368c4d12dc7d57f |
| CheckPCCPackexp0 record            | 0c94de2d11bbde3253e11d0111f73535<br>f4efc01476e20bc888747d20899bea20 |
| Accept run                         | 497c7250aa2af0858d474756575c0b74<br>6e16810cf4813f88e045b53fcff73908 |
| Final verdict record               | 891175b2493965e84dfea660cecb1c63<br>316f6ee222e45544b8f7d5c535b5e2c5 |

The file-level SHA256 checksum ledger is committed as [proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS](#). The ledger is used for the durable bundle files other than `SHA256SUMS` itself. In that ledger, `release-seal.json` has file checksum

```
1181749e236d229328b7e7723ce82294.  
47bf0b8cad191c869e04de51c6edc1cc
```

The detached checksum of `SHA256SUMS` is committed as [proof-artifacts/final-pnp-proof-report-hardened-105af51/SHA256SUMS.sha256](#) and is

```
c8b05f882d4a0c271a819dcf00916e79.  
582028d15b343fba170e3bb5cd7ef938
```

### 23.4 Final theorem extraction

The accepted proof report has theorem record

```
statement = P = NP,  
antecedent = CheckPCCPackexp(GeneratePCCPack())=accept,  
consequent = P = NP,  
conditional = true.
```

By Theorem 18.3, the accepted generated package suffices for  $P = NP$ . The final proof report records the accepted generated package checker, accepted acceptance run, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, and accepted release gate. Therefore the conditional antecedent has been discharged in the sealed artefact stack, and the theorem reported by the paper is  $P = NP$ .

### 23.5 Independent verification protocol

A reviewer can verify the final seal in two complementary modes. In artefact mode, the reviewer checks out the sealed artefact release

`final-pnp-proof-report-artifacts-hardened-105af51-sealed`

and inspects the committed bundle at

`proof-artifacts/final-pnp-proof-report-hardened-105af51/`.

The required durable files are `release-seal.json`, `SHA256SUMS`, `SHA256SUMS.sha256`, the compact artefacts under `compact/`, and the full artefacts under `full/`. File-level verification checks the committed bundle against `SHA256SUMS` while excluding any `SHA256SUMS` self-entry.

In regeneration mode, the reviewer checks that the recorded source/checker revision emits an accepted `CheckFinalPNPProofReport0` record with the same theorem statement, accepted status, and package/replay/certificate linkage. Exact equality to every committed `release-seal.json` digest is a property of the frozen artefact bundle and of reproductions run in the same clean release context; fresh regenerations in a different output context may produce different release-context digests while still preserving accepted theorem fields and stable core package, accept-run, acceptance-transcript, final-verdict, and `CheckPCCPackexp0` digests. The regeneration commands are those shown in Section 20.22. If repository access is restricted, request source and artefact access from [pnp@keaton.com.au](mailto:pnp@keaton.com.au).

## 24 Appendix A: locked NAND macro truth signatures

### 24.1 Equality macro $M_{=}$

Row order is  $(r, u, s) = 000, 001, \dots, 111$ .

| Output   | Simplified function                           | Signature |
|----------|-----------------------------------------------|-----------|
| $a_1$    | $\neg(ru)$                                    | 11111100  |
| $a_2$    | $\neg(rs)$                                    | 11111010  |
| $a_3$    | $ru$                                          | 00000011  |
| $a_4$    | $\neg(rus)$                                   | 11111110  |
| $a_5$    | $\neg r \vee u$                               | 11110011  |
| $a_6$    | $r \wedge \neg u$                             | 00001100  |
| $a_7$    | $\neg r \vee u \vee s$                        | 11110111  |
| $a_8$    | $r[(u \wedge s) \vee (\neg u \wedge \neg s)]$ | 00001001  |
| $a_9$    | $\neg r$                                      | 11110000  |
| $a_{10}$ | $r \vee \neg u$                               | 11001111  |

The positive projections are  $r = 00001111$ ,  $u = 00110011$ , and  $s = 01010101$ . The ten signatures are pairwise distinct, nonconstant, distinct from all positive projections, and essentially depend on private lock  $r$ .

### 24.2 Constant macros

For  $(r, u) = 00, 01, 10, 11$ :

| Macro | Output                  | Signature |
|-------|-------------------------|-----------|
| $M_1$ | $b_1 = \neg(ru)$        | 1110      |
| $M_1$ | $b_2 = ru$              | 0001      |
| $M_0$ | $d_1 = \neg(ru)$        | 1110      |
| $M_0$ | $d_2 = \neg r \vee u$   | 1101      |
| $M_0$ | $d_3 = r \wedge \neg u$ | 0010      |

The positive projections are  $r = 0011$  and  $u = 0101$ .

### 24.3 NAND trace-check macro $M_N$

Row order is  $(\ell, t, u, v) = 0000, 0001, \dots, 1111$ .

| Output   | Simplified function                | Signature        |
|----------|------------------------------------|------------------|
| $q_1$    | $\neg(\ell t)$                     | 1111111111110000 |
| $q_2$    | $\ell t$                           | 0000000000001111 |
| $q_3$    | $\neg(\ell u)$                     | 1111111111001100 |
| $q_4$    | $\neg(\ell v)$                     | 1111111110101010 |
| $q_5$    | $\neg(\ell t \neg u)$              | 1111111111110011 |
| $q_6$    | $\neg(\ell t u)$                   | 1111111111111100 |
| $q_7$    | $\ell t u$                         | 0000000000000011 |
| $q_8$    | $\neg(\ell t u \neg v)$            | 1111111111111101 |
| $q_9$    | $\neg \ell \vee t$                 | 1111111100001111 |
| $q_{10}$ | $\ell \wedge \neg t$               | 0000000011110000 |
| $q_{11}$ | $\neg(\ell \neg t u)$              | 1111111111001111 |
| $q_{12}$ | $\ell \neg t u$                    | 0000000000110000 |
| $q_{13}$ | $\neg(\ell \neg t u v)$            | 1111111111011111 |
| $q_{14}$ | $\ell t(\neg u \vee \neg v)$       | 0000000000001110 |
| $q_{15}$ | $\neg[\ell t(\neg u \vee \neg v)]$ | 1111111111110001 |
| $q_{16}$ | $\ell[t = \mathbf{N}(u, v)]$       | 0000000000011110 |
| $q_{17}$ | $\neg \ell$                        | 1111111100000000 |
| $q_{18}$ | $\ell \vee \neg t$                 | 1111000011111111 |

The positive projections are  $\ell = 0000000011111111$ ,  $t = 0000111100001111$ ,  $u = 0011001100110011$ ,  $v = 0101010101010101$ . The eighteen signatures are pairwise distinct, nonconstant, nonprojection, and essentially depend on  $\ell$ .

## 25 Appendix B: checklist for package acceptance

| Package  | Role                    | Formal target                                                                |
|----------|-------------------------|------------------------------------------------------------------------------|
| E        | Direct-wire verifier    | Parser, R1 to R9, full-mode obligation life-cycle, size ledger.              |
| N        | Traceable normalization | Pullback and expansion traces, materializers, witness transport.             |
| FT       | Finite table engine     | Bounded evaluator, relation evaluator, normalization transport, coverage.    |
| X        | Critical windows        | Candidate towers, Hall matching, first failed coordinate, transfer identity. |
| BC       | Branch-cycle            | Partial category, cycle and branch audits, first failed coordinates.         |
| UN       | Unary decoder           | Full-lift criterion, blocked intervals, interval audits, clean spines.       |
| HN       | Hereditary              | Shape recognizers, BWL, LN critical pairs, ParseOrExit, leaf-tightness.      |
| HResolve | Global hereditary       | H-disjoint families, LT-family audit, strong sidecar.                        |
| BUD      | Budget resolver         | Neutral grammar, envelope DP, active routing, strong sidecar.                |
| RW/FF    | Residual witnesses      | FF-LOSS, MuBridge, SpanPolicy, SaturatePositive, BCELReady.                  |
| BN2-BN6  | Cellization             | Side-tight bases, request envelopes, cancellation, shadows, hypergraphs.     |
| PkgC     | Separating consumers    | Request traces, full restoration universes, matching graphs.                 |
| Packet   | Selector seeds          | Pair, balanced-triple, and full-span spine extraction.                       |
| R        | Realizers               | Gain-only public interface, typed $\perp$ , charge surplus.                  |

|      |                   |                                                                           |
|------|-------------------|---------------------------------------------------------------------------|
| HB   | Rank negatives    | HN-BUD blocker graph, selector-silence induction.                         |
| O    | Oracle            | H/B negatives, selector silence, ZeroSlack, residual descent.             |
| G    | SAT embedding     | Macros, locked NAND threshold, residual slack at most four.               |
| PACK | Top-level checker | Interface, row keys, imports, reflection, bounds, no hidden minimization. |

## 26 Closing summary

The programme reduces SAT to exact minimization of a locked multi-output NAND direct-wire word with residual slack at most four, and proves that residual-band exact minimization is polynomial when the finite proof-carrying package accepts. The structural proof supplies compatible support replacement, charge soundness, mode projection control, finite routers, hereditary and budget closure, packet extraction, selector realization, ZeroSlack, locked NAND thresholding, and generated package sufficiency.

The final repository artefact stack discharges the generated-package condition. The accepted proof report records accepted `CheckPCCPackexp0`, accepted `CheckAcceptRun0`, accepted replay, accepted final verdict, accepted final certificate, accepted release audit, accepted final release gate, and accepted `CheckFinalPNPPProofReport0`. The final public theorem statement is

$$\text{CheckPCCPack}_{\text{exp}}(\text{GeneratePCCPack}()) = \text{accept} \implies P = NP,$$

and the sealed artefact stack records the antecedent as accepted. Hence the final conclusion of this paper is

$$P = NP.$$